

crun-cli
v1.4.0

Generated on Fri Mar 20 2026 10:50:39 for crun-cli by Doxygen 1.16.1

Fri Mar 20 2026 10:50:39

1	crun-cli	1
1.0.1	Overview	1
1.0.2	Key Features	1
1.0.3	Requirements	2
1.0.4	Usage	2
1.0.4.1	CLI Flags	2
1.0.4.2	Install Binary (optional)	2
1.0.5	Runtime Flow	2
1.0.6	Template Sources	2
1.0.7	Download	3
1.0.8	Emailware	3
1.0.9	Contributing	3
1.0.10	License	3
1.0.11	Contact	3
1.0.12	Acknowledgments	3
2	Contributor Code of Conduct	5
2.1	Our Pledge	5
2.2	Our Standards	5
2.3	Our Responsibilities	6
2.4	Scope	6
2.5	Enforcement	6
2.6	Enforcement Guidelines	6
2.7	Attribution	6
3	Contributing to crun-cli	7
3.1	Table of Contents	7
3.2	Getting Started	7
3.3	How to Contribute	8
3.4	Code Guidelines	8
3.5	Pull Request Process	8
3.6	Reporting Issues	8
3.7	Code of Conduct	9
3.8	License	9
4	Security Policy for crun-cli	11
4.1	Supported Versions	11
4.2	Reporting a Vulnerability	11
4.3	Security Practices	11
4.3.1	1. Authentication and Authorization	11
4.3.2	2. Data Encryption	11
4.3.3	3. SQL Injection Protection	12
4.3.4	4. Cross-Site Scripting (XSS) Prevention	12
4.3.5	5. Cross-Site Request Forgery (CSRF) Protection	12

4.3.6 6. Third-Party Dependency Management	12
4.3.7 7. Secure Configuration	12
4.3.8 8. Content Security Policy (CSP)	12
4.3.9 9. Monitoring and Logging	12
4.4 Security Contributions	12
4.5 Resources	12
5 Directory Hierarchy	13
5.1 Directories	13
6 Class Index	15
6.1 Class List	15
7 File Index	17
7.1 File List	17
8 Directory Documentation	19
8.1 common Directory Reference	19
8.2 inc Directory Reference	19
8.3 project Directory Reference	20
8.4 src Directory Reference	21
9 Class Documentation	23
9.1 CrunLanguage Struct Reference	23
9.1.1 Member Data Documentation	23
9.1.1.1 name	23
9.2 CrunPackage Struct Reference	23
9.2.1 Member Data Documentation	24
9.2.1.1 description	24
9.2.1.2 name	24
9.2.1.3 url	24
9.3 CrunRuntimeContext Struct Reference	24
9.3.1 Member Data Documentation	25
9.3.1.1 json_root	25
9.3.1.2 json_root_buffer	25
9.3.1.3 languages_buffer	25
9.3.1.4 packages_buffer	25
9.3.1.5 stacks_path	25
9.3.1.6 workspace_path	25
9.4 DownloadProgress Struct Reference	26
9.4.1 Member Data Documentation	26
9.4.1.1 last_percent	26
9.4.1.2 started	26
10 File Documentation	27

10.1 CODE_OF_CONDUCT.md File Reference	27
10.2 CONTRIBUTING.md File Reference	27
10.3 color.h File Reference	27
10.3.1 Detailed Description	28
10.3.2 Macro Definition Documentation	28
10.3.2.1 bgBlue	28
10.3.2.2 bgCyan	28
10.3.2.3 bgGreen	28
10.3.2.4 bgMagenta	28
10.3.2.5 bgRed	28
10.3.2.6 bgWhite	28
10.3.2.7 bgYellow	28
10.3.2.8 black	28
10.3.2.9 bleu	29
10.3.2.10 cyan	29
10.3.2.11 def	29
10.3.2.12 gray	29
10.3.2.13 green	29
10.3.2.14 magenta	29
10.3.2.15 red	29
10.3.2.16 white	29
10.3.2.17 yellow	29
10.4 color.h	29
10.5 patterns.h File Reference	30
10.5.1 Detailed Description	30
10.5.2 Macro Definition Documentation	31
10.5.2.1 CRUN_BANNER	31
10.5.2.2 CRUN_HEADER	31
10.5.2.3 HELP_MSG	31
10.5.2.4 INSTALLATION_COMPLETE_MSG	31
10.5.2.5 PROJECT_MENU	31
10.5.2.6 PROJECT_MENU_FIELD	32
10.5.2.7 PROJECT_MENU_FIELD_LENGTH	32
10.6 patterns.h	32
10.7 crun.h File Reference	32
10.7.1 Detailed Description	34
10.7.2 Macro Definition Documentation	34
10.7.2.1 __get_current_dir	34
10.7.2.2 __get_pid	34
10.7.3 Function Documentation	34
10.7.3.1 crun()	34
10.7.3.2 crun_help()	36
10.7.3.3 crun_stacks_json_checker()	36

10.7.3.4	crun_update()	37
10.7.3.5	download_crun_package()	37
10.7.3.6	free_all()	38
10.7.3.7	get_language_buffer()	38
10.7.3.8	get_packages_buffer()	39
10.7.3.9	get_user_choice()	40
10.8	crun.h	41
10.9	crun_audit.h File Reference	42
10.9.1	Detailed Description	42
10.9.2	Macro Definition Documentation	43
10.9.2.1	__mkdir	43
10.9.3	Function Documentation	43
10.9.3.1	crun_audit_error()	43
10.9.3.2	crun_audit_info()	44
10.9.3.3	crun_audit_warn()	45
10.10	crun_audit.h	45
10.11	crun_file_manager.h File Reference	45
10.11.1	Detailed Description	47
10.11.2	Macro Definition Documentation	47
10.11.2.1	CRUN_DEFAULT_SUFFIX_DIRECTORY	47
10.11.2.2	CRUN_STACKS_JSON_FILE_DEFAULT_SUFFIX	47
10.11.2.3	CRUN_STACKS_JSON_FILE_URL	47
10.11.2.4	CRUN_TMP_DIRECTORY_SUFFIX	47
10.11.2.5	OS_HOME	47
10.11.3	Function Documentation	47
10.11.3.1	copy_file_to_path()	47
10.11.3.2	get_file_data()	48
10.11.3.3	get_file_home_path()	49
10.11.3.4	get_file_size()	49
10.11.3.5	is_file_exist()	49
10.11.3.6	is_zip_archive()	50
10.12	crun_file_manager.h	51
10.13	crun_json_manager.h File Reference	51
10.13.1	Detailed Description	52
10.13.2	Function Documentation	52
10.13.2.1	get_json_object()	52
10.13.2.2	get_json_root()	53
10.14	crun_json_manager.h	54
10.15	crun_libcurl.h File Reference	54
10.15.1	Detailed Description	55
10.15.2	Macro Definition Documentation	55
10.15.2.1	CURL_CA_CERT_NAME	55
10.15.2.2	CURL_CA_CERT_REMOTE_PATH	55

10.15.3 Function Documentation	56
10.15.3.1 download_file()	56
10.15.3.2 write_data()	56
10.16 crun_libcurl.h	57
10.17 crun_zip_manager.h File Reference	57
10.17.1 Detailed Description	58
10.17.2 Macro Definition Documentation	59
10.17.2.1 MKDIR	59
10.17.2.2 PATH_SEP	59
10.17.3 Function Documentation	59
10.17.3.1 extract_zip()	59
10.18 crun_zip_manager.h	60
10.19 crun.c File Reference	60
10.19.1 Detailed Description	61
10.19.2 Function Documentation	62
10.19.2.1 crun()	62
10.19.2.2 crun_help()	63
10.19.2.3 crun_stacks_json_checker()	63
10.19.2.4 crun_update()	64
10.19.2.5 download_crun_package()	64
10.19.2.6 free_all()	65
10.19.2.7 get_language_buffer()	66
10.19.2.8 get_packages_buffer()	66
10.19.2.9 get_user_choice()	67
10.19.3 Variable Documentation	68
10.19.3.1 languages_map	68
10.19.3.2 languages_map_length	68
10.19.3.3 packages_map	68
10.19.3.4 packages_map_length	68
10.20 crun_audit.c File Reference	68
10.20.1 Detailed Description	69
10.20.2 Function Documentation	69
10.20.2.1 crun_audit_error()	69
10.20.2.2 crun_audit_info()	70
10.20.2.3 crun_audit_warn()	71
10.21 crun_file_manager.c File Reference	71
10.21.1 Detailed Description	72
10.21.2 Macro Definition Documentation	72
10.21.2.1 CRUN_IO_BUFFER_SIZE	72
10.21.3 Function Documentation	72
10.21.3.1 copy_file_to_path()	72
10.21.3.2 get_file_data()	73
10.21.3.3 get_file_home_path()	74

10.21.3.4 <code>get_file_size()</code>	74
10.21.3.5 <code>is_file_exist()</code>	74
10.21.3.6 <code>is_zip_archive()</code>	75
10.22 <code>crun_json_manager.c</code> File Reference	76
10.22.1 Detailed Description	76
10.22.2 Function Documentation	76
10.22.2.1 <code>get_json_object()</code>	76
10.22.2.2 <code>get_json_root()</code>	77
10.23 <code>crun_libcurl.c</code> File Reference	78
10.23.1 Detailed Description	79
10.23.2 Function Documentation	79
10.23.2.1 <code>download_file()</code>	79
10.23.2.2 <code>write_data()</code>	80
10.24 <code>crun_zip_manager.c</code> File Reference	80
10.24.1 Detailed Description	81
10.24.2 Function Documentation	81
10.24.2.1 <code>extract_zip()</code>	81
10.25 <code>main.c</code> File Reference	82
10.25.1 Detailed Description	83
10.25.2 Function Documentation	83
10.25.2.1 <code>main()</code>	83
10.26 <code>README.md</code> File Reference	84
10.27 <code>SECURITY.md</code> File Reference	84
Index	85

Chapter 1

crun-cli

Crun CLI

C-based project scaffolding CLI

[Overview](#) • [Key Features](#) • [Requirements](#) • [Usage](#) • [Runtime Flow](#) • [Template Sources](#) • [Download](#) • [Emailware](#) • [Contributing](#) • [License](#) • [Contact](#) • [Acknowledgments](#)

1.0.1 Overview

crun-cli fetches template metadata, lets you choose a language/template, downloads or reuses cached packages, then either extracts a ZIP scaffold (with optional init script) or installs a direct single-file template into your current directory.

1.0.2 Key Features

- Interactive language/template menus
- Template cache in `~/.local/share/crun/`
- Hidden extraction workspace in `~/.local/share/crun/tmp/`
- Optional init execution:
 - `__init__.sh` (Unix)
 - `__init__.bat` (Windows)
- Init script cleanup after execution
- ZIP scaffold copy excludes zip artifacts
- Single-file template install when package is not a ZIP
- Timestamped audit logs in CLI and `~/.local/share/crun/audit.log`

1.0.3 Requirements

- [CMake](#)
- [Ninja](#)
- [libcurl](#)
- [cJSON](#)
- [minizip](#)

1.0.4 Usage

```
git clone https://github.com/zouari-oss/crun-cli
cd crun-cli/project

./run rb  # configure
./run i   # build
./run r   # run crun
```

1.0.4.1 CLI Flags

```
./bin/build/crun --help
./bin/build/crun --update
```

- `--help` prints CLI usage and notes.
- `--update` refreshes local `crun_stacks.json` from the remote source.

1.0.4.2 Install Binary (optional)

```
sudo cp bin/build/crun /usr/local/bin/crun
```

1.0.5 Runtime Flow

1. Load `crun_stacks.json` (download if missing)
2. Select language and template
3. Resolve template package (cache hit or download)
4. Detect package type (ZIP vs single file)
5. For ZIP: extract into hidden temp workspace
6. For ZIP: run init script if present
7. For ZIP: remove `__init__.sh` / `__init__.bat`
8. For ZIP: copy output to current directory (excluding `*.zip`)
9. For single file: copy file directly into current directory
10. Clean temporary workspace (ZIP path only)

1.0.6 Template Sources

- [cpkg templates](#)

1.0.7 Download

You can [download](#) the latest installable version of crun-cli for Windows, macOS and Linux.

1.0.8 Emailware

crun-cli is an emailware. Meaning, if you liked using this app or it has helped you in any way, would like you send as an email at zouariomar20@gmail.com about anything you'd want to say about this software. I'd really appreciate it!

1.0.9 Contributing

Contributions are welcome! Please feel free to submit a Pull Request.

1.0.10 License

This repository is licensed under the GPL-3.0 License. You are free to use, modify, and distribute the content. See the LICENSE file for details.

1.0.11 Contact

For questions or suggestions, feel free to reach out:

- GitHub: [ZouariOmar](#)
- Email: zouariomar20@gmail.com
- LinkedIn: [Zouari Omar](#)

1.0.12 Acknowledgments

Built with [for](#) for the open-source community.

Chapter 2

Contributor Code of Conduct

2.1 Our Pledge

In the interest of fostering an open and welcoming environment, we as contributors and maintainers pledge to make participation in the crun-cli project and our community a harassment-free experience for everyone, regardless of age, body size, disability, ethnicity, gender identity and expression, level of experience, nationality, personal appearance, race, religion, or sexual identity and orientation.

2.2 Our Standards

Examples of behavior that contribute to a positive environment include:

- Demonstrating respect and consideration for others
- Providing constructive and respectful feedback
- Acknowledging different perspectives and experiences
- Showing empathy towards other community members
- Being open to learning from constructive criticism

Examples of unacceptable behavior include:

- The use of sexualized language or imagery, and unwelcome sexual attention or advances
- Trolling, insulting or derogatory comments, and personal or political attacks
- Public or private harassment
- Publishing others' private information, such as a physical or electronic address, without explicit permission
- Other conduct which could reasonably be considered inappropriate in a professional setting

2.3 Our Responsibilities

Project maintainers are responsible for clarifying standards of acceptable behavior and are expected to take appropriate and fair corrective action in response to any instances of unacceptable behavior.

Maintainers have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that do not align with this Code of Conduct, as well as to ban temporarily or permanently any contributor for other behaviors that they deem inappropriate, threatening, offensive, or harmful.

2.4 Scope

This Code of Conduct applies within project spaces (such as GitHub repositories, issue trackers, and pull requests) and in public spaces when an individual is representing the project or its community. Examples of representing the project include using an official project email address, posting via an official social media account, or acting as an appointed representative at an online or offline event.

2.5 Enforcement

Instances of abusive, harassing, or otherwise unacceptable behavior may be reported by contacting the project team at zouariomar20@gmail.com. All complaints will be reviewed and investigated promptly and fairly.

All project team members are obligated to respect the privacy and security of the reporter of any incident.

2.6 Enforcement Guidelines

Project maintainers will follow these guidelines to determine the consequences for any actions deemed unacceptable:

1. Correction
Impact: A private, written warning from project maintainers, clarifying the nature of the violation and expectations for future behavior.
2. Warning
Impact: A warning with consequences for continued inappropriate behavior. The user may be temporarily blocked from interactions in the project space.
3. Temporary Ban
Impact: A temporary ban from engaging with the project, lasting a defined period.
4. Permanent Ban
Impact: A permanent ban from any interaction within the project space.

2.7 Attribution

This Code of Conduct is adapted from the [Contributor Covenant](https://www.contributor-covenant.org/version/2/1/code_of_conduct.html), version 2.1, available at https://www.contributor-covenant.org/version/2/1/code_of_conduct.html.

For answers to common questions about this code of conduct, see the [FAQ](https://www.contributor-covenant.org/translations). Translations are available at <https://www.contributor-covenant.org/translations>.

Chapter 3

Contributing to crun-cli

Thank you for your interest in contributing to crun-cli! Contributions from the community are welcome and appreciated.

This document outlines the process for contributing, whether you're reporting bugs, suggesting enhancements, improving documentation, or submitting code.

3.1 Table of Contents

- Getting Started
- How to Contribute
- Code Guidelines
- Pull Request Process
- Reporting Issues
- Code of Conduct
- License

3.2 Getting Started

1. Fork the repository.
2. Clone your fork to your local machine:
`git clone https://github.com/YOUR_USERNAME/crun-cli.git`
3. Create a new branch for your feature or fix:
`git checkout -b your-branch-name`
4. Make your changes and commit them.
5. Push to your fork and submit a pull request.

3.3 How to Contribute

You can contribute in many ways:

- Fix bugs or typos
- Improve documentation
- Suggest new features
- Submit new code
- Write tests
- Review pull requests

Please check the [issues](#) tab to see where help is needed.

3.4 Code Guidelines

- Keep your code clean and readable.
- Follow the coding style used in the project.
- Write descriptive commit messages.
- Include tests for new functionality if applicable.
- Make sure your code builds and passes all tests before submitting a PR.

3.5 Pull Request Process

1. Ensure your changes pass existing tests and do not introduce regressions.
2. Ensure your branch is up to date with the [main](#) or dev branch.
3. Submit a pull request with a clear description of your changes.
4. Wait for review and respond to feedback.

3.6 Reporting Issues

When reporting bugs or issues, please include:

- A clear and descriptive title.
- Steps to reproduce the issue.
- Expected vs. actual behavior.
- Screenshots or logs if available.
- Your environment (OS, version, etc.).

3.7 Code of Conduct

Please be respectful and constructive in all interactions.

We follow a [Code of Conduct](#) to ensure a welcoming community for everyone.

3.8 License

By contributing to crun-cli, you agree that your contributions will be licensed under the same license as the project.

Thank you again for your interest in contributing!

Chapter 4

Security Policy for crun-cli

4.1 Supported Versions

The crun-cli project provides security updates for the following versions:

Version	Supported
1.↔ x.x	:white_check_↔ mark:
0.↔ x.x	:↔ x:↔

4.2 Reporting a Vulnerability

If you discover a security vulnerability in crun-cli, we encourage responsible disclosure. To report a vulnerability:

1. Contact us via email at zouariomar20@gmail.com with the details.
2. Please provide a detailed description of the issue, including steps to reproduce it and, if available, a proof of concept.
3. We aim to respond within 48 hours and will work closely with you to resolve the issue.

We appreciate the security community's efforts in helping us maintain crun-cli's safety and integrity.

4.3 Security Practices

crun-cli follows industry best practices for security to ensure safe transactions and protect user data:

4.3.1 1. Authentication and Authorization

- crun-cli uses secure methods for user authentication, including strong password hashing (e.g., bcrypt).
- Access control is implemented to restrict unauthorized actions.

4.3.2 2. Data Encryption

- All sensitive data is encrypted in transit (via HTTPS/TLS) and at rest, including user credentials and personal information.

4.3.3 3. SQL Injection Protection

- Parameterized queries and prepared statements are used throughout to prevent SQL injection.
- ORM (Object-Relational Mapping) is leveraged where possible.

4.3.4 4. Cross-Site Scripting (XSS) Prevention

- User inputs are sanitized and validated to prevent XSS attacks, and user-generated content is escaped to avoid code injection.

4.3.5 5. Cross-Site Request Forgery (CSRF) Protection

- CSRF tokens are included in forms and API requests to secure user actions.

4.3.6 6. Third-Party Dependency Management

- Dependencies are regularly scanned for vulnerabilities with tools such as [OWASP Dependency-Check](#).
- Critical security patches for dependencies are applied promptly.

4.3.7 7. Secure Configuration

- Default settings follow secure configurations, and sensitive data is stored securely (e.g., .env files for environment-specific settings).

4.3.8 8. Content Security Policy (CSP)

- A strict CSP is enforced to limit content sources, reducing the risk of XSS and unauthorized resource loading.

4.3.9 9. Monitoring and Logging

- Logs are maintained for key operations and monitored for abnormal patterns. Logs avoid storing sensitive information.

4.4 Security Contributions

Contributors can help enhance crun-cli's security by following these practices:

- Adhere to secure coding standards and the above security practices when contributing.
- Conduct thorough security testing before submitting a pull request.
- Avoid including any sensitive data in code (e.g., passwords, API keys).

4.5 Resources

For additional guidance on secure development, refer to:

- [OWASP Secure Coding Practices Checklist](#)
- [CWE Top 25 Most Dangerous Software Errors](#)

Thank you for helping us make crun-cli secure for our users and the community!

Chapter 5

Directory Hierarchy

5.1 Directories

common	19
color.h	27
patterns.h	30
inc	19
crun.h	32
crun_audit.h	42
crun_file_manager.h	45
crun_json_manager.h	51
crun_libcurl.h	54
crun_zip_manager.h	57
project	20
common	19
color.h	27
patterns.h	30
inc	19
crun.h	32
crun_audit.h	42
crun_file_manager.h	45
crun_json_manager.h	51
crun_libcurl.h	54
crun_zip_manager.h	57
src	21
crun.c	60
crun_audit.c	68
crun_file_manager.c	71
crun_json_manager.c	76
crun_libcurl.c	78
crun_zip_manager.c	80
main.c	82
src	21
crun.c	60
crun_audit.c	68
crun_file_manager.c	71
crun_json_manager.c	76
crun_libcurl.c	78
crun_zip_manager.c	80
main.c	82

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CrunLanguage	23
CrunPackage	23
CrunRuntimeContext	24
DownloadProgress	26

Chapter 7

File Index

7.1 File List

Here is a list of all files with brief descriptions:

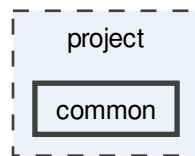
color.h	ANSI Color header file	27
patterns.h	Patterns header file	30
crun.h	Core crun CLI interfaces	32
crun_audit.h	Audit logging API for CLI/file outputs	42
crun_file_manager.h	File/path utilities for crun runtime data	45
crun_json_manager.h	JSON parsing helpers for stacks metadata	51
crun_libcurl.h	Network download helpers built on top of libcurl	54
crun_zip_manager.h	ZIP extraction helpers	57
crun.c	Crun source file	60
crun_audit.c	Crun audit/logging helpers implementation	68
crun_file_manager.c	Crun_file_manager source file	71
crun_json_manager.c	Crun_json_manager source file	76
crun_libcurl.c	Crun_libcurl source file	78
crun_zip_manager.c	Crun_zip_manager source file	80
main.c	Main source file	82

Chapter 8

Directory Documentation

8.1 common Directory Reference

Directory dependency graph for common:



Files

- file [color.h](#)
ANSI Color header file.
- file [patterns.h](#)
patterns header file

8.2 inc Directory Reference

Directory dependency graph for inc:

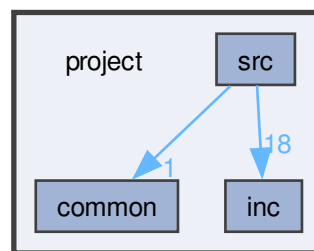


Files

- file [crun.h](#)
Core crun CLI interfaces.
- file [crun_audit.h](#)
Audit logging API for CLI/file outputs.
- file [crun_file_manager.h](#)
File/path utilities for crun runtime data.
- file [crun_json_manager.h](#)
JSON parsing helpers for stacks metadata.
- file [crun_libcurl.h](#)
Network download helpers built on top of libcurl.
- file [crun_zip_manager.h](#)
ZIP extraction helpers.

8.3 project Directory Reference

Directory dependency graph for project:

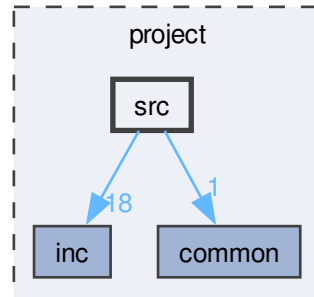


Directories

- directory [common](#)
- directory [inc](#)
- directory [src](#)

8.4 src Directory Reference

Directory dependency graph for src:



Files

- file [crun.c](#)
crun source file
- file [crun_audit.c](#)
crun audit/logging helpers implementation
- file [crun_file_manager.c](#)
crun_file_manager source file
- file [crun_json_manager.c](#)
crun_json_manager source file
- file [crun_libcurl.c](#)
crun_libcurl source file
- file [crun_zip_manager.c](#)
crun_zip_manager source file
- file [main.c](#)
Main source file.

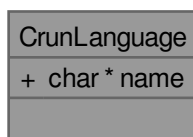
Chapter 9

Class Documentation

9.1 CrunLanguage Struct Reference

#include <crun.h>

Collaboration diagram for CrunLanguage:



Public Attributes

- char * [name](#)

9.1.1 Member Data Documentation

9.1.1.1 name

char* CrunLanguage::name

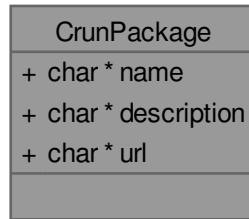
The documentation for this struct was generated from the following file:

- [crun.h](#)

9.2 CrunPackage Struct Reference

#include <crun.h>

Collaboration diagram for CrunPackage:



Public Attributes

- char * [name](#)
- char * [description](#)
- char * [url](#)

9.2.1 Member Data Documentation

9.2.1.1 description

char * CrunPackage::description

9.2.1.2 name

char* CrunPackage::name

9.2.1.3 url

char * CrunPackage::url

The documentation for this struct was generated from the following file:

- [crun.h](#)

9.3 CrunRuntimeContext Struct Reference

```
#include <crun.h>
```


Collaboration diagram for CrunRuntimeContext:

CrunRuntimeContext
+ char * languages_buffer + char * packages_buffer + const char * json_root_buffer + const char * stacks_path + const char * workspace_path + cJSON * json_root

Public Attributes

- char * [languages_buffer](#)
- char * [packages_buffer](#)
- const char * [json_root_buffer](#)
- const char * [stacks_path](#)
- const char * [workspace_path](#)
- cJSON * [json_root](#)

9.3.1 Member Data Documentation

9.3.1.1 json_root

cJSON* CrunRuntimeContext::json_root

9.3.1.2 json_root_buffer

const char* CrunRuntimeContext::json_root_buffer

9.3.1.3 languages_buffer

char* CrunRuntimeContext::languages_buffer

9.3.1.4 packages_buffer

char* CrunRuntimeContext::packages_buffer

9.3.1.5 stacks_path

const char* CrunRuntimeContext::stacks_path

9.3.1.6 workspace_path

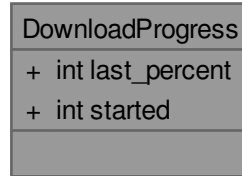
const char* CrunRuntimeContext::workspace_path

The documentation for this struct was generated from the following file:

- [crun.h](#)

9.4 DownloadProgress Struct Reference

Collaboration diagram for DownloadProgress:



Public Attributes

- int [last_percent](#)
- int [started](#)

9.4.1 Member Data Documentation

9.4.1.1 last_percent

int DownloadProgress::last_percent

9.4.1.2 started

int DownloadProgress::started

The documentation for this struct was generated from the following file:

- [crun_libcurl.c](#)

Chapter 10

File Documentation

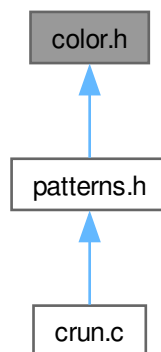
10.1 CODE_OF_CONDUCT.md File Reference

10.2 CONTRIBUTING.md File Reference

10.3 color.h File Reference

ANSI Color header file.

This graph shows which files directly or indirectly include this file:



Macros

- `#define red "\x1B[1;31m"`
- `#define green "\x1B[1;32m"`
- `#define yellow "\x1B[1;33m"`
- `#define bleu "\x1B[1;34m"`
- `#define cyan "\x1B[1;36m"`
- `#define def "\x1B[0m"`
- `#define black "\x1B[1;30m"`
- `#define magenta "\x1B[1;35m"`
- `#define white "\x1B[1;37m"`
- `#define gray "\x1B[0;37m"`
- `#define bgRed "\x1B[41m"`
- `#define bgGreen "\x1B[42m"`

- `#define bgYellow "\x1B[43m"`
- `#define bgBlue "\x1B[44m"`
- `#define bgMagenta "\x1B[45m"`
- `#define bgCyan "\x1B[46m"`
- `#define bgWhite "\x1B[47m"`

10.3.1 Detailed Description

ANSI Color header file.

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.1

Date

2025-01-10

Copyright

Copyright (c) 2025

`cpkg`

10.3.2 Macro Definition Documentation

10.3.2.1 bgBlue

```
#define bgBlue "\x1B[44m"
```

10.3.2.2 bgCyan

```
#define bgCyan "\x1B[46m"
```

10.3.2.3 bgGreen

```
#define bgGreen "\x1B[42m"
```

10.3.2.4 bgMagenta

```
#define bgMagenta "\x1B[45m"
```

10.3.2.5 bgRed

```
#define bgRed "\x1B[41m"
```

10.3.2.6 bgWhite

```
#define bgWhite "\x1B[47m"
```

10.3.2.7 bgYellow

```
#define bgYellow "\x1B[43m"
```

10.3.2.8 black

```
#define black "\x1B[1;30m"
```

10.3.2.9 bleu

```
#define bleu "\x1B[1;34m"
```

10.3.2.10 cyan

```
#define cyan "\x1B[1;36m"
```

10.3.2.11 def

```
#define def "\x1B[0m"
```

10.3.2.12 gray

```
#define gray "\x1B[0;37m"
```

10.3.2.13 green

```
#define green "\x1B[1;32m"
```

10.3.2.14 magenta

```
#define magenta "\x1B[1;35m"
```

10.3.2.15 red

```
#define red "\x1B[1;31m"
```

10.3.2.16 white

```
#define white "\x1B[1;37m"
```

10.3.2.17 yellow

```
#define yellow "\x1B[1;33m"
```

10.4 color.h

[Go to the documentation of this file.](#)

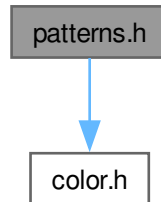
```
00001
00011
00012 #ifndef __COLOR_H__
00013 #define __COLOR_H__
00014
00015 // #####
00016 // ### HEADERS & MACROS DECLARATION PART ###
00017 // #####
00018
00019 // Text colors
00020 #define red "\x1B[1;31m"
00021 #define green "\x1B[1;32m"
00022 #define yellow "\x1B[1;33m"
00023 #define bleu "\x1B[1;34m"
00024 #define cyan "\x1B[1;36m"
00025 #define def "\x1B[0m"
00026 #define black "\x1B[1;30m"
00027 #define magenta "\x1B[1;35m"
00028 #define white "\x1B[1;37m"
00029 #define gray "\x1B[0;37m"
00030
00031 // Background colors
00032 #define bgRed "\x1B[41m"
00033 #define bgGreen "\x1B[42m"
00034 #define bgYellow "\x1B[43m"
00035 #define bgBlue "\x1B[44m"
00036 #define bgMagenta "\x1B[45m"
00037 #define bgCyan "\x1B[46m"
00038 #define bgWhite "\x1B[47m"
00039
00040 #endif // __COLOR_H__
```

10.5 patterns.h File Reference

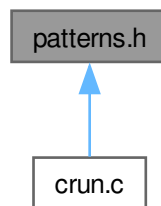
patterns header file

```
#include "color.h"
```

Include dependency graph for patterns.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define [CRUN_HEADER](#)
- #define [CRUN_BANNER](#)
- #define [PROJECT_MENU_FIELD_LENGTH](#)(i, field)
- #define [PROJECT_MENU_FIELD](#)(buff, i, field)
- #define [PROJECT_MENU](#)(fileds)
- #define [HELP_MSG](#)
- #define [INSTALLATION_COMPLETE_MSG](#) printf("%sInstallation Complete!\nHAPPY COD-ING :)\n%s", [green](#), [def](#))

10.5.1 Detailed Description

patterns header file

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.1

Date

2024-10-19

Copyright

Copyright (c) 2024

patterns.h

10.5.2 Macro Definition Documentation

10.5.2.1 CRUN_BANNER

```
#define CRUN_BANNER
```

Value:

```
printf("%s\t\t\t\t\t==== Crun Project " \
      "\t\t\t\t\t====\n\t\t\t\t\t==== By " \
      "@ZouariOmar \t\t\t\t\t==== Visit Us In: " \
      "https://github.com/zouari-oss ====%s\n\n",
```

10.5.2.6 PROJECT_MENU_FIELD

```
#define PROJECT_MENU_FIELD(
    buff,
    i,
    field)
```

Value:

```
sprintf(buff, "\t\t\t[%d] %s\n", i, field)
```

10.5.2.7 PROJECT_MENU_FIELD_LENGTH

```
#define PROJECT_MENU_FIELD_LENGTH(
    i,
    field)
```

Value:

```
snprintf(NULL, 0, "\t\t\t[%d] %s\n", i, field)
```

10.6 patterns.h

[Go to the documentation of this file.](#)

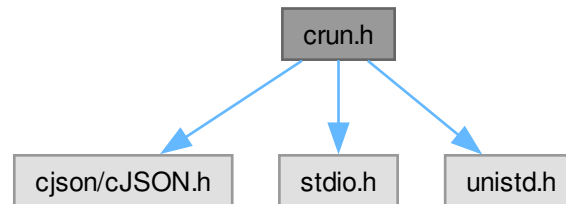
```
00001
00011
00012 // #####
00013 // ### HEADERS & MACROS DECLARATION PART ###
00014 // #####
00015
00016 #include "color.h"
00017
00018 #ifdef _WIN32
00019 #define CRUN_HEADER \
00020     system("cmd /c echo [Crun - V1.4]");
00021 #else
00022 #define CRUN_HEADER
00023     system("echo \"\033[0;32m$(figlet -w $(tput cols) ' \
00024         \"      Crun - V1.4')\033[0m\"")
00025 #endif
00026
00027 #define CRUN_BANNER
00028     printf("%s\t\t\t===== Crun Project \"
00029         \"=====\\n\t\t\t===== By \"
00030         \"@ZouariOmar =====\\n\t\t\t===== Visit Us In: \" \
00031         \"https://github.com/zouari-oss =====%s\\n\",
00032         bleu, def)
00033
00034 #define PROJECT_MENU_FIELD_LENGTH(i, field) snprintf(NULL, 0, "\t\t\t[%d] %s\n", i, field)
00035
00036 #define PROJECT_MENU_FIELD(buff, i, field) sprintf(buff, "\t\t\t[%d] %s\n", i, field)
00037
00038 #define PROJECT_MENU(files)
00039     printf("%s\t\t\t-----\\n\" \
00040         \"%s\t\t\tProject Language:\\n\"
00041         \"%s%s\"
00042         \"%\t\t\t[0] Exit\\n\"
00043         \"%s\t\t\tVisit https://github.com/zouari-oss/crun-cli for more info\\n\"
00044         \"%s\t\t\t-----\\n\" \
00045         \"...%s\",
00046         yellow, red, yellow, fields, red, yellow, def)
00047
00048 #define HELP_MSG
00049     printf("%scrun usage:%s\\n\"
00050         \"    crun          Run interactive template selection flow\\n\"
00051         \"    crun --help | -h    Show this help message\\n\"
00052         \"    crun --update      Refresh local stacks metadata (crun_stacks.json)\\n\\n\" \
00053         \"%sNotes:%s\\n\"
00054         \"    - Templates are cached under ~/.local/share/crun/ (Linux).\\n\"
00055         \"    - Audit logs are written to ~/.local/share/crun/audit.log.\\n\",
00056         yellow, def, yellow, def)
00057
00058 #define INSTALLATION_COMPLETE_MSG printf("%sInstallation Complete!\\nHAPPY CODING :)\\n%s\", green, def)
```

10.7 crun.h File Reference

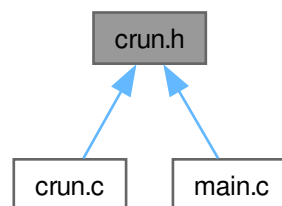
Core crun CLI interfaces.


```
#include <cjson/cJSON.h>
#include <stdio.h>
#include <unistd.h>
```

Include dependency graph for crun.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [CrunLanguage](#)
- struct [CrunPackage](#)
- struct [CrunRuntimeContext](#)

Macros

- `#define` [__get_current_dir](#) `getcwd`
- `#define` [__get_pid](#) `getpid`

Functions

- void [crun](#) ()
Main CLI flow entrypoint.
- void [crun_help](#) ()
Print command-line usage/help.
- int [crun_update](#) ()
Refresh local stacks metadata from remote source.
- void [crun_stacks_json_checker](#) (const char *)
Ensure stacks metadata file exists locally.

- void [get_user_choice](#) (int *, const char *, const int)
Prompt for menu choice with validation.
- const char * [download_crun_package](#) (const struct [CrunPackage](#) *)
Resolve package archive from cache or download it.
- char * [get_language_buffer](#) (cJSON *)
Build selectable language menu buffer from JSON root.
- char * [get_packages_buffer](#) (cJSON *, const char *)
Build selectable package menu buffer for one language.
- void [free_all](#) (void **, const size_t)
Release language/package maps and additional owned pointers.

10.7.1 Detailed Description

Core crun CLI interfaces.

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.3

Date

2026-03-18

Copyright

Copyright (c) 2025

[crun.h](#)

10.7.2 Macro Definition Documentation

10.7.2.1 `__get_current_dir`

```
#define __get_current_dir getcwd
```

10.7.2.2 `__get_pid`

```
#define __get_pid getpid
```

10.7.3 Function Documentation

10.7.3.1 `crun()`

```
void crun ()
```

Main CLI flow entrypoint.

10.7.3.2 crun_help()

```
void crun_help ()
```

Print command-line usage/help.

Here is the caller graph for this function:



10.7.3.3 crun_stacks_json_checker()

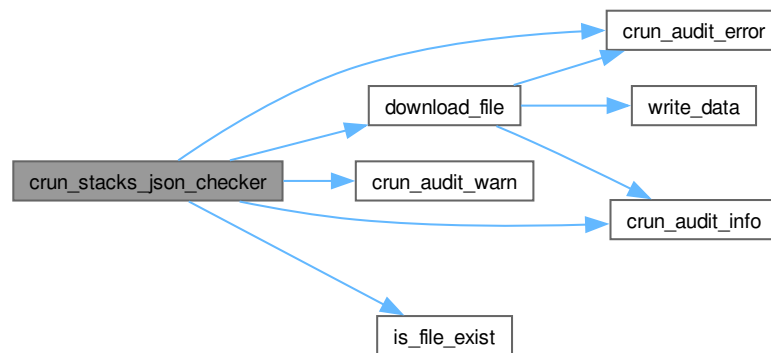
```
void crun_stacks_json_checker (
    const char * crun_stacks_json_file_path)
```

Ensure stacks metadata file exists locally.

Parameters

crun_stacks_json_file_path	Absolute path to local metadata file.
----------------------------	---------------------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



10.7.3.4 `crun_update()`

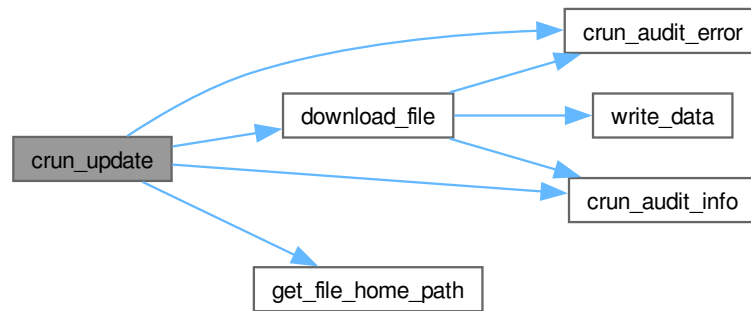
```
int crun_update ()
```

Refresh local stacks metadata from remote source.

Returns

`int` `EXIT_SUCCESS` on success, `EXIT_FAILURE` otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:

10.7.3.5 `download_crun_package()`

```
const char * download_crun_package (
    const struct CrunPackage * crun_package_map)
```

Resolve package archive from cache or download it.

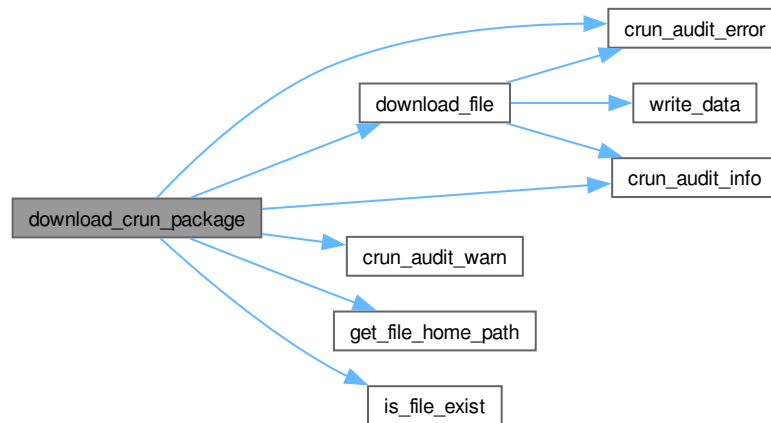
Parameters

<code>crun_package_map</code>	Selected package metadata.
-------------------------------	----------------------------

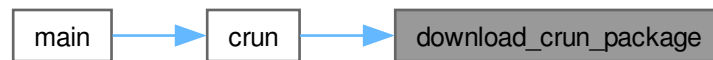
Returns

const char* Allocated absolute path to package archive, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.7.3.6 free_all()

```
void free_all (
    void ** ptrs,
    const size_t len)
```

Release language/package maps and additional owned pointers.

Parameters

ptrs	Extra pointers to free.
len	Number of pointers in ptrs.

10.7.3.7 get_language_buffer()

```
char * get_language_buffer (
    cJSON * root)
```

Build selectable language menu buffer from JSON root.

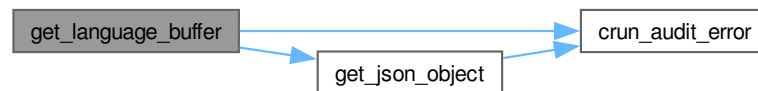
Parameters

root	Parsed JSON root.
------	-------------------

Returns

char* Allocated menu text buffer, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.7.3.8 get_packages_buffer()

```
char * get_packages_buffer (
    cJSON * root,
    const char * language_name)
```

Build selectable package menu buffer for one language.

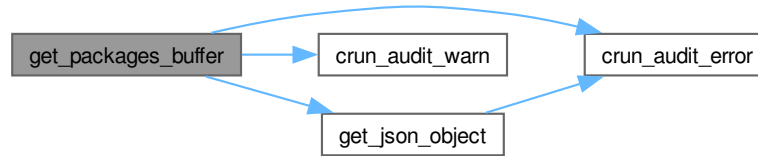
Parameters

root	Parsed JSON root.
language_name	Selected language key.

Returns

char* Allocated menu text buffer, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.7.3.9 get__user__choice()

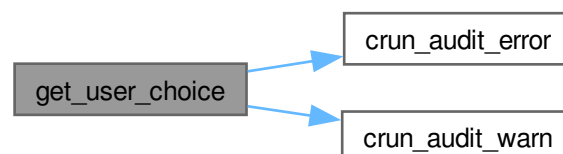
```
void get__user__choice (
    int * user_choice,
    const char * menu,
    const int limiter)
```

Prompt for menu choice with validation.

Parameters

user_choice	Output selected value.
menu	Rendered menu text.
limiter	Upper bound (exclusive), usually map length.

Here is the call graph for this function:



Here is the caller graph for this function:



10.8 crun.h

[Go to the documentation of this file.](#)

```

00001
00011
00012 #ifndef __CRUN_H__
00013 #define __CRUN_H__
00014
00015 // #####
00016 // ##### HEADERS & MACROS DECLARATION PART #####
00017 // #####
00018
00019 #include <cjson/cJSON.h>
00020 #include <stdio.h>
00021
00022 #ifdef _WIN32
00023 #include <direct.h>
00024 #include <process.h>
00025 #define __get_current_dir __getcwd
00026 #define __get_pid __getpid
00027
00028 #else
00029 #include <unistd.h>
00030 #define __get_current_dir getcwd
00031 #define __get_pid getpid
00032 #endif
00033
00034 // #####
00035 // ##### STRUCTS DECLARATION PART #####
00036 // #####
00037
00038 struct CrunLanguage {
00039     char *name;
00040 }; // CrunLanguage Structure
00041
00042 struct CrunPackage {
00043     char *name, *description, *url;
00044 }; // CrunPackage Structure
00045
00046 struct CrunRuntimeContext {
00047     char *languages_buffer;
00048     char *packages_buffer;
00049     const char *json_root_buffer;
00050     const char *stacks_path;
00051     const char *workspace_path;
00052     cJSON *json_root;
00053 }; // CrunRuntimeContext Structure
00054
00055 // #####
00056 // ##### FUNCTIONS DECLARATION PART #####
00057 // #####
00058
00062 void crun();
00063
00067 void crun_help();
00068
00074 int crun_update();
00075
00081 void crun_stacks_json_checker(const char *);
00082
00090 void get_user_choice(int *, const char *, const int);
00091
00098 const char *download_crun_package(const struct CrunPackage *);
00099
00106 char *get_language_buffer(cJSON *);
00107
00115 char *get_packages_buffer(cJSON *, const char *);

```

```

00116
00123 void free_all(void **, const size_t);
00124
00125 #endif // __CRUN_H__

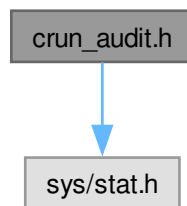
```

10.9 crun_audit.h File Reference

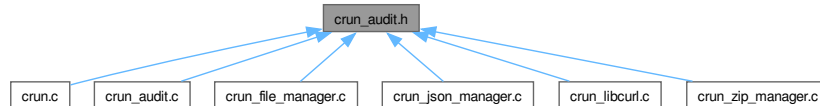
Audit logging API for CLI/file outputs.

#include <sys/stat.h>

Include dependency graph for crun_audit.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define `__mkdir`(path)

Functions

- void `crun_audit_info` (const char *,...)

Emit an informational audit event.
- void `crun_audit_warn` (const char *,...)

Emit a warning audit event.
- void `crun_audit_error` (const char *,...)

Emit an error audit event.

10.9.1 Detailed Description

Audit logging API for CLI/file outputs.

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.2

Date

2026-03-18

Copyright

Copyright (c) 2025

[crun_audit.h](#)

10.9.2 Macro Definition Documentation

10.9.2.1 `__mkdir`

```
#define __mkdir(  
                path)
```

Value:

`mkdir(path, 0755)`

10.9.3 Function Documentation

10.9.3.1 `crun_audit_error()`

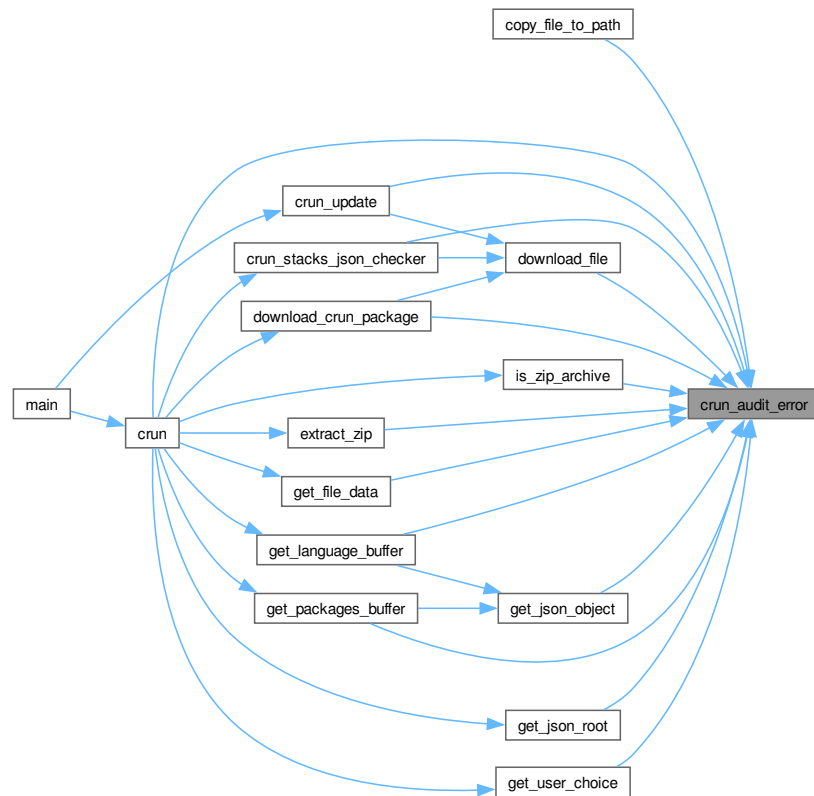
```
void crun_audit_error (  
                        const char * fmt,  
                        ...)
```

Emit an error audit event.

Parameters

fmt	printf-style message format.
...	Format arguments.

Here is the caller graph for this function:



10.9.3.2 crun_audit_info()

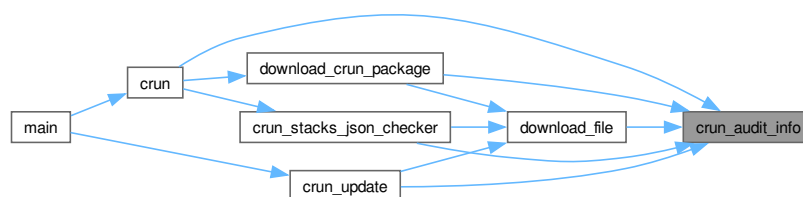
```
void crun_audit_info (
    const char * fmt,
    ...)
```

Emit an informational audit event.

Parameters

fmt	printf-style message format.
...	Format arguments.

Here is the caller graph for this function:



10.9.3.3 crun_audit_warn()

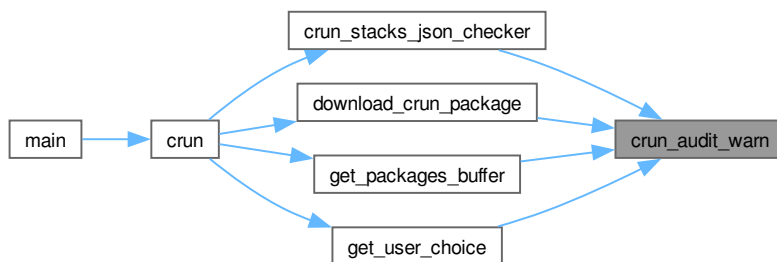
```
void crun_audit_warn (
    const char * fmt,
    ...)
```

Emit a warning audit event.

Parameters

fmt	printf-style message format.
...	Format arguments.

Here is the caller graph for this function:



10.10 crun_audit.h

[Go to the documentation of this file.](#)

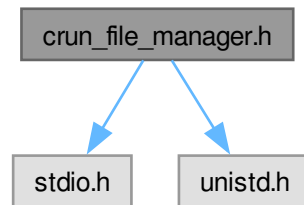
```

00001
00011
00012 #ifndef __CRUN_AUDIT_H__
00013 #define __CRUN_AUDIT_H__
00014
00015 // #####
00016 // ### HEADERS & MACROS DECLARATION PART ###
00017 // #####
00018
00019 #ifdef _WIN32
00020 #include <direct.h>
00021 #define __mkdir(path) _mkdir(path)
00022
00023 #else
00024 #include <sys/stat.h>
00025 #define __mkdir(path) mkdir(path, 0755)
00026 #endif
00027
00028 // #####
00029 // ### FUNCTIONS DECLARATION PART ###
00030 // #####
00031
00038 void crun_audit_info(const char *, ...);
00039
00046 void crun_audit_warn(const char *, ...);
00047
00054 void crun_audit_error(const char *, ...);
00055
00056 #endif // __CRUN_AUDIT_H__
```

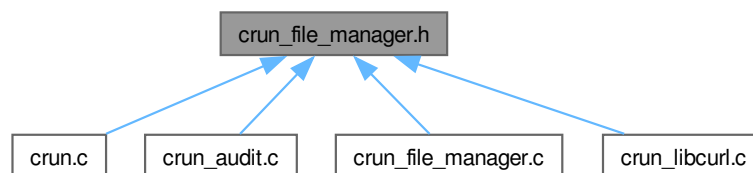
10.11 crun_file_manager.h File Reference

File/path utilities for crun runtime data.

```
#include <stdio.h>
#include <unistd.h>
Include dependency graph for crun_file_manager.h:
```



This graph shows which files directly or indirectly include this file:



Macros

- `#define OS_HOME` `getenv("HOME")`
- `#define CRUN_DEFAULT_SUFFIX_DIRECTORY` `"/.local/share/crun/"`
- `#define CRUN_STACKS_JSON_FILE_URL` `"https://raw.githubusercontent.com/ZouariOmar/cpkg/refs/heads/main/crun/crun_stacks.json"`
- `#define CRUN_STACKS_JSON_FILE_DEFAULT_SUFFIX` `CRUN_DEFAULT_SUFFIX_DIRECTORY "crun_stacks.json"`
- `#define CRUN_TMP_DIRECTORY_SUFFIX` `CRUN_DEFAULT_SUFFIX_DIRECTORY "tmp"`

Functions

- `char * get_file_data (const char *)`
Read full file content into a null-terminated buffer.
- `int is_file_exist (const char *)`
Check if file or directory exists.
- `const long get_file_size (FILE *)`
Compute total file size for opened stream.
- `const char * get_file_home_path (const char *)`
Build home-based absolute path using suffix.
- `int is_zip_archive (const char *)`
Detect whether a file is a ZIP archive using its signature.

- int `copy_file_to_path` (const char *, const char *)
Copy a file from source path to destination path.

10.11.1 Detailed Description

File/path utilities for crun runtime data.

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.3

Date

2026-03-18

Copyright

Copyright (c) 2025

`crun_file_manager.h`

10.11.2 Macro Definition Documentation

10.11.2.1 CRUN_DEFAULT_SUFFIX_DIRECTORY

```
#define CRUN_DEFAULT_SUFFIX_DIRECTORY "/.local/share/crun/"
```

10.11.2.2 CRUN_STACKS_JSON_FILE_DEFAULT_SUFFIX

```
#define CRUN_STACKS_JSON_FILE_DEFAULT_SUFFIX CRUN_DEFAULT_SUFFIX_DIRECTORY "crun_↵  
stacks.json"
```

10.11.2.3 CRUN_STACKS_JSON_FILE_URL

```
#define CRUN_STACKS_JSON_FILE_URL "https://raw.githubusercontent.com/ZouariOmar/cpkg/refs/heads/↵  
main/crun/crun_stacks.json"
```

10.11.2.4 CRUN_TMP_DIRECTORY_SUFFIX

```
#define CRUN_TMP_DIRECTORY_SUFFIX CRUN_DEFAULT_SUFFIX_DIRECTORY "tmp"
```

10.11.2.5 OS_HOME

```
#define OS_HOME getenv("HOME")
```

10.11.3 Function Documentation

10.11.3.1 copy_file_to_path()

```
int copy_file_to_path (  
    const char * source_path,  
    const char * target_path)
```

Copy a file from source path to destination path.

Parameters

source_path	Existing file path.
-------------	---------------------

target_path	Destination file path.
-------------	------------------------

Returns

int EXIT_SUCCESS on success, EXIT_FAILURE otherwise.

Here is the call graph for this function:



10.11.3.2 get_file_data()

```
char * get_file_data (  
    const char * filename)
```

Read full file content into a null-terminated buffer.

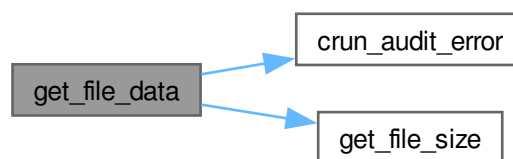
Parameters

filename	Absolute/relative file path.
----------	------------------------------

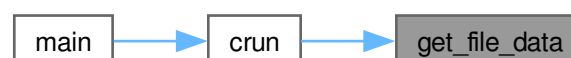
Returns

char* Allocated buffer (caller frees), or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.11.3.3 get_file_home_path()

```
const char * get_file_home_path (
    const char * suffix)
```

Build home-based absolute path using suffix.

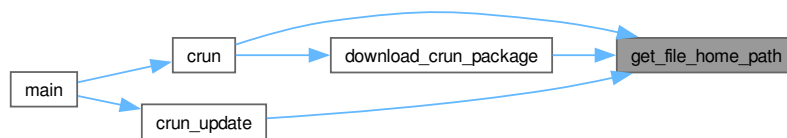
Parameters

suffix	Path suffix appended to OS home location.
--------	---

Returns

const char* Allocated path buffer (caller frees), or NULL on failure.

Here is the caller graph for this function:



10.11.3.4 get_file_size()

```
const long get_file_size (
    FILE * fp)
```

Compute total file size for opened stream.

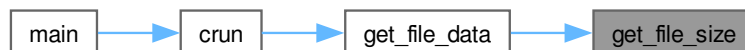
Parameters

fp	Opened file pointer.
----	----------------------

Returns

long File size in bytes, -1 on failure.

Here is the caller graph for this function:



10.11.3.5 is_file_exist()

```
int is_file_exist (
    const char * filename)
```

Check if file or directory exists.

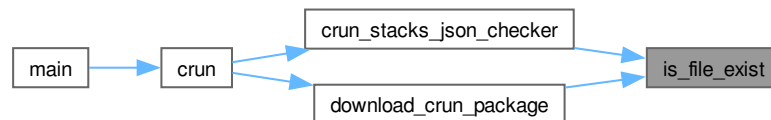
Parameters

filename	Path to check.
----------	----------------

Returns

int Non-zero if path exists, zero otherwise.

Here is the caller graph for this function:

10.11.3.6 `is_zip_archive()`

```
int is_zip_archive (
    const char * filename)
```

Detect whether a file is a ZIP archive using its signature.

Parameters

filename	Path to the file to inspect.
----------	------------------------------

Returns

int Non-zero if ZIP signature is detected, zero otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



10.12 crun_file_manager.h

[Go to the documentation of this file.](#)

```

00001
00011
00012 #ifndef __CRUN_FILE_MANAGER_H__
00013 #define __CRUN_FILE_MANAGER_H__
00014
00015 // #####
00016 // ### HEADERS & MACROS DECLARATION PART ###
00017 // #####
00018
00019 #include <stdio.h>
00020
00021 #ifdef _WIN32
00022 #include <io.h>
00023 #define OS_HOME getenv("APPDATA")
00024 #define F_OK 0
00025 #define access _access
00026 #define CRUN_DEFAULT_SUFFIX_DIRECTORY "\\crun\\"
00027
00028 #else
00029 #include <unistd.h>
00030 #define OS_HOME getenv("HOME")
00031 #define CRUN_DEFAULT_SUFFIX_DIRECTORY "/.local/share/crun/"
00032 #endif
00033
00034 #define CRUN_STACKS_JSON_FILE_URL
00035 "https://raw.githubusercontent.com/ZouariOmar/cpkg/refs/heads/main/crun/crun_stacks.json"
00036 #define CRUN_STACKS_JSON_FILE_DEFAULT_SUFFIX CRUN_DEFAULT_SUFFIX_DIRECTORY
00037 "crun_stacks.json"
00038 #define CRUN_TMP_DIRECTORY_SUFFIX CRUN_DEFAULT_SUFFIX_DIRECTORY "tmp"
00039
00040 // #####
00041 // ### FUNCTIONS DECLARATION PART ###
00042 // #####
00043
00044 char *get_file_data(const char *);
00045
00046 int is_file_exist(const char *);
00047
00048 const long get_file_size(FILE *);
00049
00050 const char *get_file_home_path(const char *);
00051
00052 int is_zip_archive(const char *);
00053
00054 int copy_file_to_path(const char *, const char *);
00055
00056 #endif // __CRUN_FILE_MANAGER_H__

```

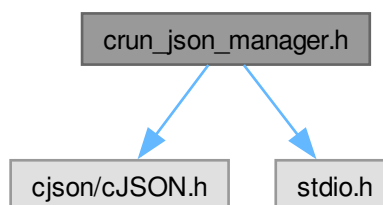
10.13 crun_json_manager.h File Reference

JSON parsing helpers for stacks metadata.

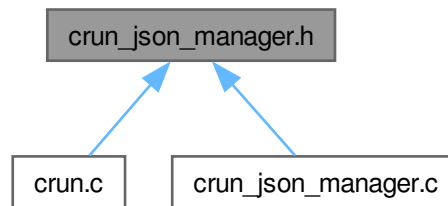
```
#include <cjson/cJSON.h>
```

```
#include <stdio.h>
```

Include dependency graph for crun_json_manager.h:



This graph shows which files directly or indirectly include this file:



Functions

- cJSON * [get_json_root](#) (const char *json_buffer)
Parse JSON buffer into cJSON root object.
- cJSON * [get_json_object](#) (cJSON *root, const char *obj_name)
Retrieve required JSON object child by name.

10.13.1 Detailed Description

JSON parsing helpers for stacks metadata.

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.3

Date

2026-03-18

Copyright

Copyright (c) 2025

[crun_json_manager.h](#)

10.13.2 Function Documentation

10.13.2.1 get_json_object()

```
cJSON * get_json_object (
    cJSON * root,
    const char * obj_name)
```

Retrieve required JSON object child by name.

Parameters

root	Parent JSON root.
obj_name	Object key name.

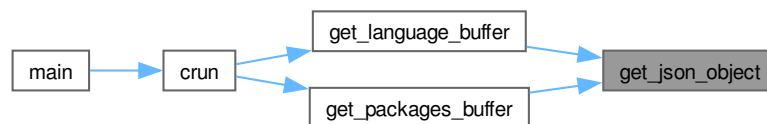
Returns

cJSON* Object node, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.13.2.2 get_json_root()

cJSON * get_json_root (
 const char * json_buffer)
Parse JSON buffer into cJSON root object.

Parameters

json_buffer	JSON text buffer.
-------------	-------------------

Returns

cJSON* Parsed root object, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.14 crun_json_manager.h

[Go to the documentation of this file.](#)

```

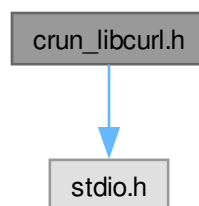
00001
00011
00012 #ifndef __CRUN_JSON_MANAGER_H__
00013 #define __CRUN_JSON_MANAGER_H__
00014
00015 // #####
00016 // ### HEADERS & MACROS DECLARATION PART ###
00017 // #####
00018
00019 #include <cjson/cJSON.h>
00020 #include <stdio.h>
00021
00022 // #####
00023 // ### FUNCTIONS DECLARATION PART ###
00024 // #####
00025
00032 cJSON *get_json_root(const char *json_buffer);
00033
00041 cJSON *get_json_object(cJSON *root, const char *obj_name);
00042
00043 #endif // __CRUN_JSON_MANAGER_H__
  
```

10.15 crun_libcurl.h File Reference

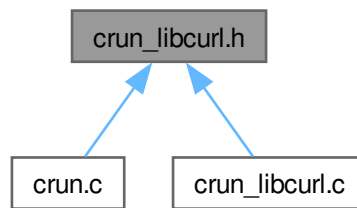
Network download helpers built on top of libcurl.

`#include <stdio.h>`

Include dependency graph for `crun_libcurl.h`:



This graph shows which files directly or indirectly include this file:



Macros

- `#define` [CURL_CA_CERT_NAME](#) "cacert-2026-03-19.pem"
- `#define` [CURL_CA_CERT_REMOTE_PATH](#) "https://curl.se/ca/" [CURL_CA_CERT_NAME](#)

Functions

- `size_t` [write_data](#) (void *, size_t, size_t, void *)
libcurl write callback used when storing response data to a file.
- `int` [download_file](#) (const char *, const char *)
Download a file from URL to a destination path.

10.15.1 Detailed Description

Network download helpers built on top of libcurl.

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.3

Date

2026-03-18

Copyright

Copyright (c) 2025

[crun_libcurl.h](#)

10.15.2 Macro Definition Documentation

10.15.2.1 CURL_CA_CERT_NAME

```
#define CURL_CA_CERT_NAME "cacert-2026-03-19.pem"
```

10.15.2.2 CURL_CA_CERT_REMOTE_PATH

```
#define CURL_CA_CERT_REMOTE_PATH "https://curl.se/ca/" CURL_CA_CERT_NAME
```

10.15.3 Function Documentation

10.15.3.1 download_file()

```
int download_file (
```

```
    const char * url,  
    const char * out)
```

Download a file from URL to a destination path.

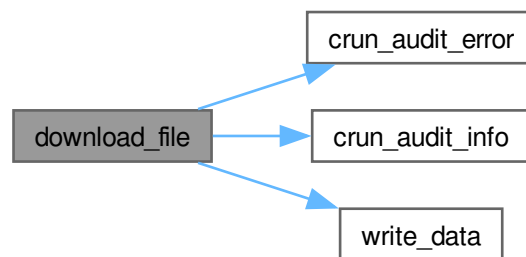
Parameters

url	Remote URL.
out	Destination file path.

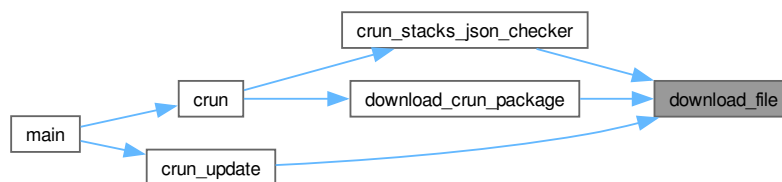
Returns

int EXIT_SUCCESS on success, EXIT_FAILURE otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



10.15.3.2 write_data()

```
size_t write_data (
```

```
    void * ptr,  
    size_t size,  
    size_t nmemb,  
    void * stream)
```


libcurl write callback used when storing response data to a file.

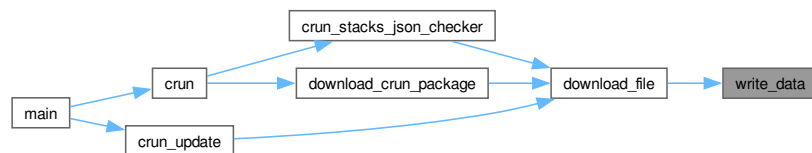
Parameters

ptr	Pointer to downloaded bytes.
size	Size of each element.
nmemb	Number of elements.
stream	Destination stream.

Returns

size_t Number of bytes written.

Here is the caller graph for this function:



10.16 crun_libcurl.h

[Go to the documentation of this file.](#)

```

00001
00011
00012 #ifndef __CRUN_LIBCURL_H__
00013 #define __CRUN_LIBCURL_H__
00014
00015 // #####
00016 // ### HEADERS & MACROS DECLARATION PART ###
00017 // #####
00018
00019 #define CURL_CA_CERT_NAME "cacert-2026-03-19.pem"
00020 #define CURL_CA_CERT_REMOTE_PATH "https://curl.se/ca/" CURL_CA_CERT_NAME
00021
00022 #include <stdio.h>
00023
00024 // #####
00025 // ### FUNCTIONS DECLARATION PART ###
00026 // #####
00027
00037 size_t write_data(void *, size_t, size_t, void *);
00038
00046 int download_file(const char *, const char *);
00047
00048 #endif // __CRUN_LIBCURL_H__

```

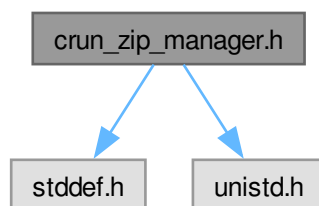
10.17 crun_zip_manager.h File Reference

ZIP extraction helpers.

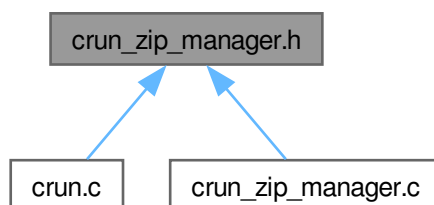
```
#include <stddef.h>
```

```
#include <unistd.h>
```

Include dependency graph for `crun_zip_manager.h`:



This graph shows which files directly or indirectly include this file:



Macros

- `#define MKDIR(path)`
- `#define PATH_SEP '/'`

Functions

- `int extract_zip(const char *, const char *, char *, size_t)`
Extract ZIP content safely into output directory.

10.17.1 Detailed Description

ZIP extraction helpers.

Author

ZouariOmar zouariomar20@gmail.com

Version

0.2

Date

2026-03-18

Copyright

Copyright (c) 2025

[crun_zip_manager.h](#)

10.17.2 Macro Definition Documentation

10.17.2.1 MKDIR

```
#define MKDIR(  
    path)
```

Value:

mkdir(path, 0755)

10.17.2.2 PATH_SEP

```
#define PATH_SEP '/'
```

10.17.3 Function Documentation

10.17.3.1 extract_zip()

```
int extract_zip (  
    const char * zip_path,  
    const char * out_dir,  
    char * init_script_path,  
    size_t init_script_path_size)
```

Extract ZIP content safely into output directory.

Detects init script path (___init__.sh or ___init__.bat) while extracting.

Parameters

zip_path	Source zip file path.
out_dir	Destination extraction directory.
init_script_path	Optional output buffer to receive detected init script path.
init_script_path_size	Size of init_script_path buffer.

Returns

int EXIT_SUCCESS on success, EXIT_FAILURE otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



10.18 crun_zip_manager.h

[Go to the documentation of this file.](#)

```

00001
00011
00012 #ifndef __CRUN_ZIP_MANAGER_H__
00013 #define __CRUN_ZIP_MANAGER_H__
00014
00015 // #####
00016 // ### HEADERS & MACROS DECLARATION PART ###
00017 // #####
00018
00019 #include <stddef.h>
00020
00021 #ifdef _WIN32
00022 #include <direct.h>
00023 #define MKDIR(path) _mkdir(path)
00024 #define PATH_SEP '\\\\'
00025
00026 #else
00027 #include <unistd.h>
00028 #define MKDIR(path) mkdir(path, 0755)
00029 #define PATH_SEP '/'
00030 #endif
00031
00032 // #####
00033 // ### FUNCTIONS DECLARATION PART ###
00034 // #####
00035
00047 int extract_zip(const char *, const char *, char *, size_t);
00048
00049 #endif // __CRUN_ZIP_MANAGER_H__

```

10.19 crun.c File Reference

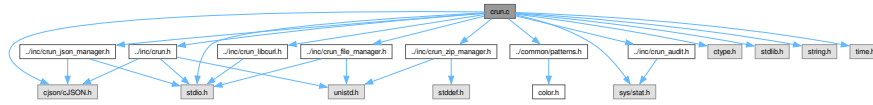
crun source file

```

#include "../inc/crun.h"
#include "../common/patterns.h"
#include "../inc/crun_audit.h"
#include "../inc/crun_file_manager.h"
#include "../inc/crun_json_manager.h"
#include "../inc/crun_libcurl.h"
#include "../inc/crun_zip_manager.h"
#include <cjson/cJSON.h>
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>

```

Include dependency graph for crun.c:



Functions

- void [crun_help](#) ()
Print command-line usage/help.
- int [crun_update](#) ()
Refresh local stacks metadata from remote source.
- void [crun](#) ()
Main CLI flow entrypoint.
- const char * [download_crun_package](#) (const struct [CrunPackage](#) *crun_package_map)
Resolve package archive from cache or download it.
- void [get_user_choice](#) (int *user_choice, const char *menu, const int limiter)
Prompt for menu choice with validation.
- void [crun_stacks_json_checker](#) (const char *crun_stacks_json_file_path)
Ensure stacks metadata file exists locally.
- char * [get_language_buffer](#) (cJSON *root)
Build selectable language menu buffer from JSON root.
- char * [get_packages_buffer](#) (cJSON *root, const char *language_name)
Build selectable package menu buffer for one language.
- void [free_all](#) (void **ptrs, size_t len)
Release language/package maps and additional owned pointers.

Variables

- struct [CrunLanguage](#) * [languages_map](#) = NULL
- size_t [languages_map_length](#) = 1
- struct [CrunPackage](#) * [packages_map](#) = NULL
- size_t [packages_map_length](#) = 1

10.19.1 Detailed Description

crun source file

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.2

Date

2025-11-26

Copyright

Copyright (c) 2025

[crun.c](#)

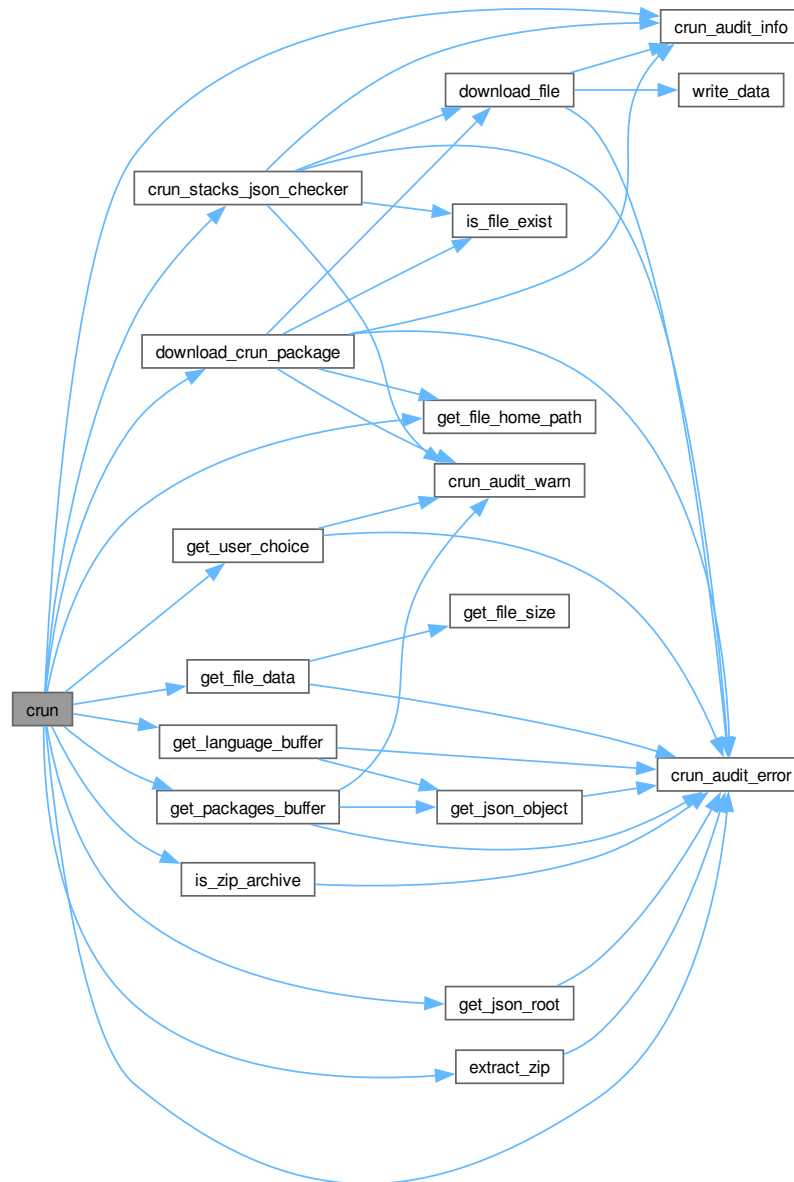
10.19.2 Function Documentation

10.19.2.1 crun()

void crun ()

Main CLI flow entrypoin.

Here is the call graph for this function:



Here is the caller graph for this function:

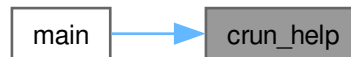


10.19.2.2 `crun_help()`

```
void crun_help ()
```

Print command-line usage/help.

Here is the caller graph for this function:



10.19.2.3 `crun_stacks_json_checker()`

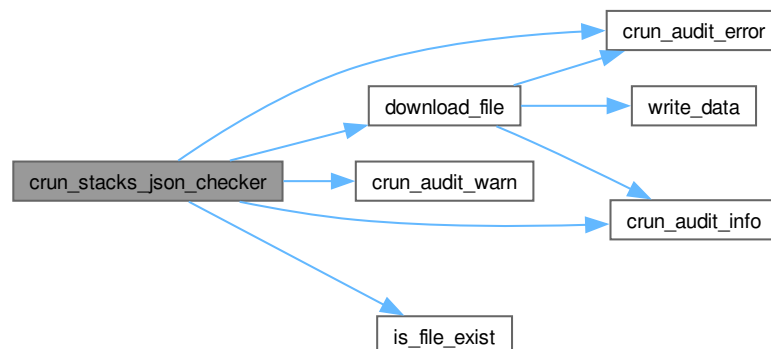
```
void crun_stacks_json_checker (  
    const char * crun_stacks_json_file_path)
```

Ensure stacks metadata file exists locally.

Parameters

<code>crun_stacks_json_file_path</code>	Absolute path to local metadata file.
---	---------------------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



10.19.2.4 crun_update()

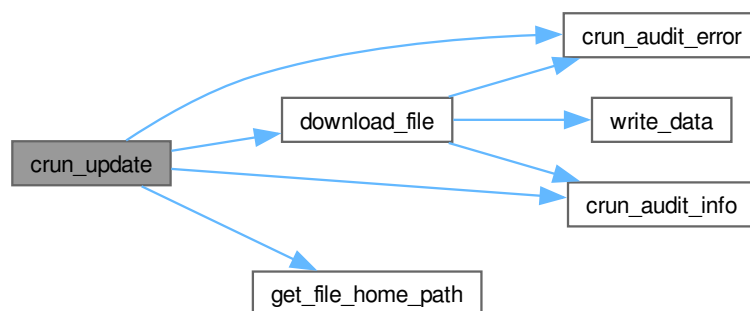
int crun_update ()

Refresh local stacks metadata from remote source.

Returns

int EXIT_SUCCESS on success, EXIT_FAILURE otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



10.19.2.5 download_crun_package()

const char * download_crun_package (

const struct [CrunPackage](#) * crun_package_map)

Resolve package archive from cache or download it.

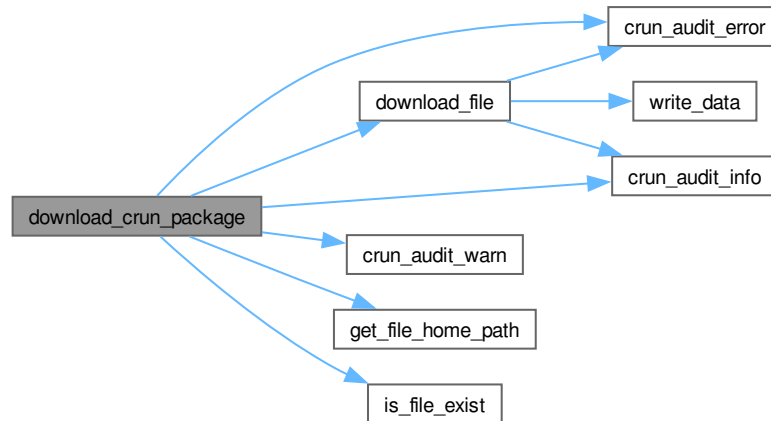
Parameters

<code>crun_package_map</code>	Selected package metadata.
-------------------------------	----------------------------

Returns

`const char*` Allocated absolute path to package archive, or `NULL` on failure.

Here is the call graph for this function:



Here is the caller graph for this function:

10.19.2.6 `free_all()`

```
void free_all (
    void ** ptrs,
    const size_t len)
```

Release language/package maps and additional owned pointers.

Parameters

<code>ptrs</code>	Extra pointers to free.
<code>len</code>	Number of pointers in <code>ptrs</code> .

10.19.2.7 `get_language_buffer()`

```
char * get_language_buffer (
    cJSON * root)
```

Build selectable language menu buffer from JSON root.

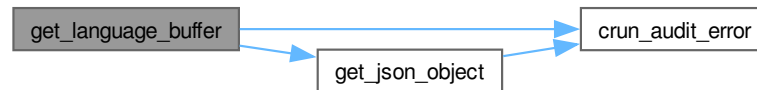
Parameters

root	Parsed JSON root.
------	-------------------

Returns

char* Allocated menu text buffer, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.19.2.8 `get_packages_buffer()`

```
char * get_packages_buffer (
    cJSON * root,
    const char * language_name)
```

Build selectable package menu buffer for one language.

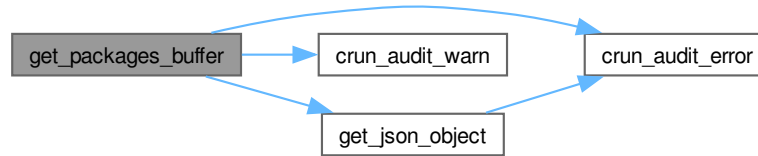
Parameters

root	Parsed JSON root.
language_name	Selected language key.

Returns

char* Allocated menu text buffer, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.19.2.9 get__user__choice()

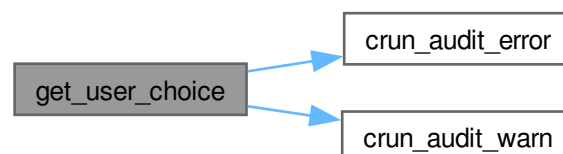
```
void get__user__choice (
    int * user__choice,
    const char * menu,
    const int limiter)
```

Prompt for menu choice with validation.

Parameters

user__choice	Output selected value.
menu	Rendered menu text.
limiter	Upper bound (exclusive), usually map length.

Here is the call graph for this function:



Here is the caller graph for this function:



10.19.3 Variable Documentation

10.19.3.1 languages__map

struct [CrunLanguage](#)* languages__map = NULL

10.19.3.2 languages__map__length

size_t languages__map__length = 1

10.19.3.3 packages__map

struct [CrunPackage](#)* packages__map = NULL

10.19.3.4 packages__map__length

size_t packages__map__length = 1

10.20 crun__audit.c File Reference

crun audit/logging helpers implementation

```
#include "../inc/crun_audit.h"
```

```
#include "../inc/crun_file_manager.h"
```

```
#include <stdarg.h>
```

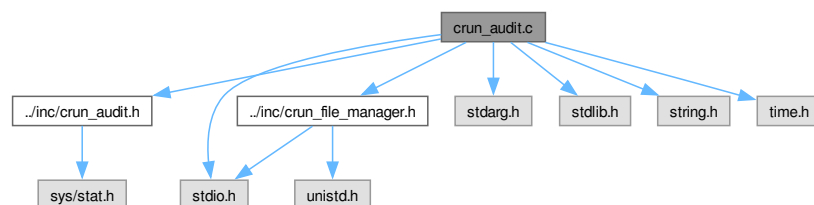
```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <time.h>
```

Include dependency graph for crun__audit.c:



Functions

- void [crun_audit_info](#) (const char *fmt,...)
Emit an informational audit event.

- void `crun_audit_warn` (const char *fmt,...)
Emit a warning audit event.
- void `crun_audit_error` (const char *fmt,...)
Emit an error audit event.

10.20.1 Detailed Description

crun audit/logging helpers implementation

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.2

Date

2025-11-26

Copyright

Copyright (c) 2025

`crun_audit.c`

10.20.2 Function Documentation

10.20.2.1 `crun_audit_error()`

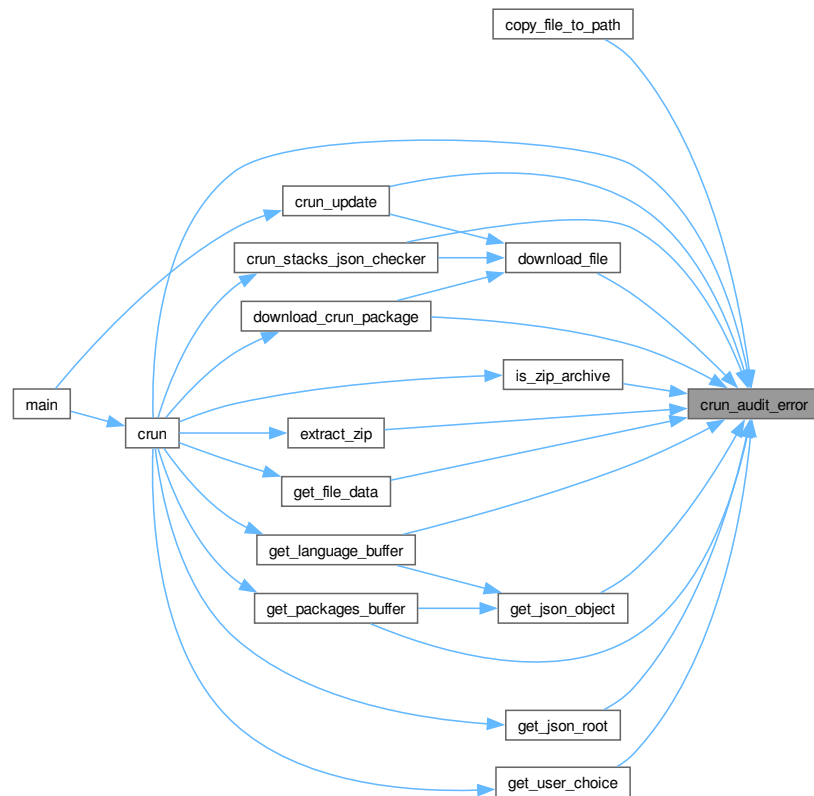
```
void crun_audit_error (  
    const char * fmt,  
    ...)
```

Emit an error audit event.

Parameters

fmt	printf-style message format.
...	Format arguments.

Here is the caller graph for this function:



10.20.2.2 crun_audit_info()

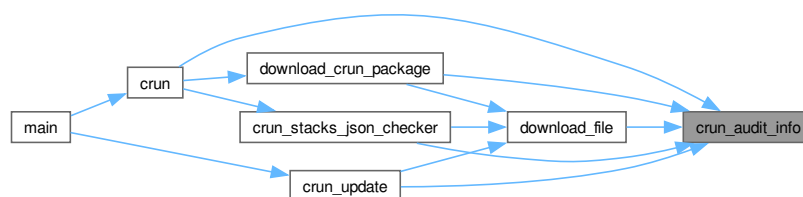
```
void crun_audit_info (
    const char * fmt,
    ...)
```

Emit an informational audit event.

Parameters

fmt	printf-style message format.
...	Format arguments.

Here is the caller graph for this function:



10.20.2.3 crun_audit_warn()

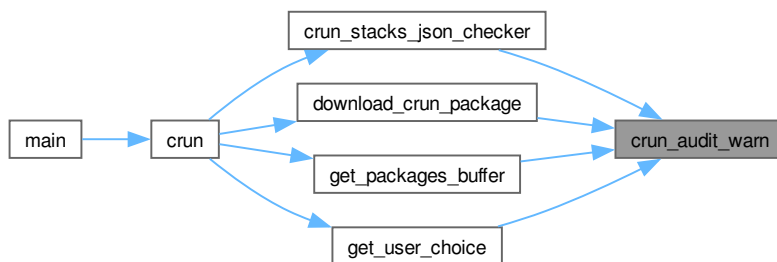
```
void crun_audit_warn (
    const char * fmt,
    ...)
```

Emit a warning audit event.

Parameters

fmt	printf-style message format.
...	Format arguments.

Here is the caller graph for this function:



10.21 crun_file_manager.c File Reference

crun_file_manager source file

```
#include "../inc/crun_file_manager.h"
```

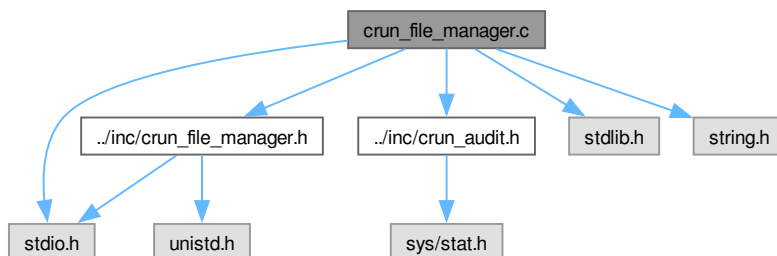
```
#include "../inc/crun_audit.h"
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

Include dependency graph for `crun_file_manager.c`:



Macros

- `#define CRUN_IO_BUFFER_SIZE 8192`

Functions

- char * [get_file_data](#) (const char *filename)
Read full file content into a null-terminated buffer.
- int [is_file_exist](#) (const char *filename)
Check if file or directory exists.
- const char * [get_file_home_path](#) (const char *suffix)
Build home-based absolute path using suffix.
- const long [get_file_size](#) (FILE *fp)
Compute total file size for opened stream.
- int [is_zip_archive](#) (const char *filename)
Detect whether a file is a ZIP archive using its signature.
- int [copy_file_to_path](#) (const char *source_path, const char *target_path)
Copy a file from source path to destination path.

10.21.1 Detailed Description

crun_file_manager source file

Author

ZouariOmar zouariomar20@gmail.com

Version

0.1

Date

2025-12-26

Copyright

Copyright (c) 2025

[crun_file_manager.c](#)

10.21.2 Macro Definition Documentation

10.21.2.1 CRUN_IO_BUFFER_SIZE

```
#define CRUN_IO_BUFFER_SIZE 8192
```

10.21.3 Function Documentation

10.21.3.1 copy_file_to_path()

```
int copy_file_to_path (
    const char * source_path,
    const char * target_path)
```

Copy a file from source path to destination path.

Parameters

source_path	Existing file path.
target_path	Destination file path.

Returns

int EXIT_SUCCESS on success, EXIT_FAILURE otherwise.

Here is the call graph for this function:



10.21.3.2 get_file_data()

```
char * get_file_data (  
    const char * filename)
```

Read full file content into a null-terminated buffer.

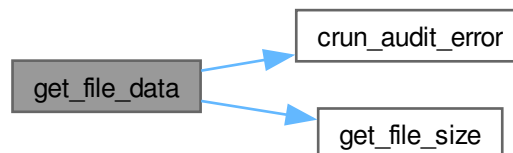
Parameters

filename	Absolute/relative file path.
----------	------------------------------

Returns

char* Allocated buffer (caller frees), or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.21.3.3 `get_file_home_path()`

```
const char * get_file_home_path (
    const char * suffix)
```

Build home-based absolute path using suffix.

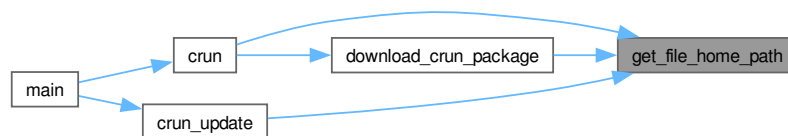
Parameters

suffix	Path suffix appended to OS home location.
--------	---

Returns

const char* Allocated path buffer (caller frees), or NULL on failure.

Here is the caller graph for this function:



10.21.3.4 `get_file_size()`

```
const long get_file_size (
    FILE * fp)
```

Compute total file size for opened stream.

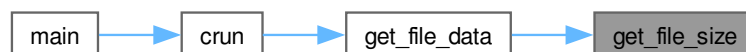
Parameters

fp	Opened file pointer.
----	----------------------

Returns

long File size in bytes, -1 on failure.

Here is the caller graph for this function:



10.21.3.5 `is_file_exist()`

```
int is_file_exist (
    const char * filename)
```

Check if file or directory exists.

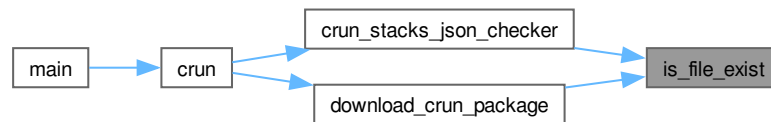
Parameters

filename	Path to check.
----------	----------------

Returns

int Non-zero if path exists, zero otherwise.

Here is the caller graph for this function:



10.21.3.6 is_zip_archive()

```
int is_zip_archive (  
    const char * filename)
```

Detect whether a file is a ZIP archive using its signature.

Parameters

filename	Path to the file to inspect.
----------	------------------------------

Returns

int Non-zero if ZIP signature is detected, zero otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



10.22 crun_json_manager.c File Reference

crun_json_manager source file

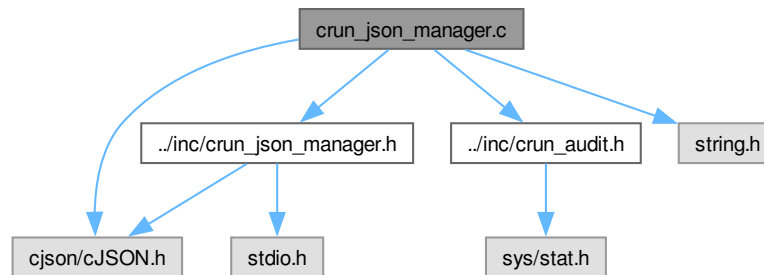
```
#include "../inc/crun_json_manager.h"
```

```
#include "../inc/crun_audit.h"
```

```
#include <cjson/cJSON.h>
```

```
#include <string.h>
```

Include dependency graph for crun_json_manager.c:



Functions

- `cJSON *` [`get\_json\_root`](#) (`const char *``json_bufferer`)
Parse JSON buffer into cJSON root object.
- `cJSON *` [`get\_json\_object`](#) (`cJSON *``root`, `const char *``obj_name`)
Retrieve required JSON object child by name.

10.22.1 Detailed Description

crun_json_manager source file

Author

ZouariOmar zouariomar20@gmail.com

Version

0.1

Date

2025-12-25

Copyright

Copyright (c) 2025

[crun_json_manager.c](#)

10.22.2 Function Documentation

10.22.2.1 `get_json_object()`

```
cJSON * get_json_object (
    cJSON * root,
    const char * obj_name)
```

Retrieve required JSON object child by name.

Parameters

root	Parent JSON root.
obj_name	Object key name.

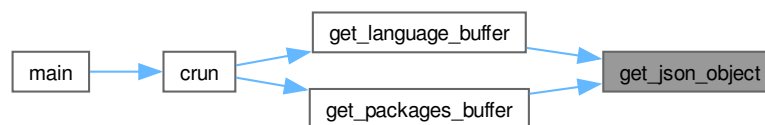
Returns

cJSON* Object node, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



10.22.2.2 get__json__root()

cJSON * get__json__root (
 const char * json_buffer)
Parse JSON buffer into cJSON root object.

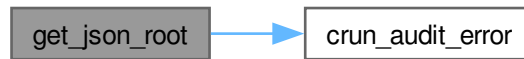
Parameters

json_buffer	JSON text buffer.
-------------	-------------------

Returns

cJSON* Parsed root object, or NULL on failure.

Here is the call graph for this function:



Here is the caller graph for this function:



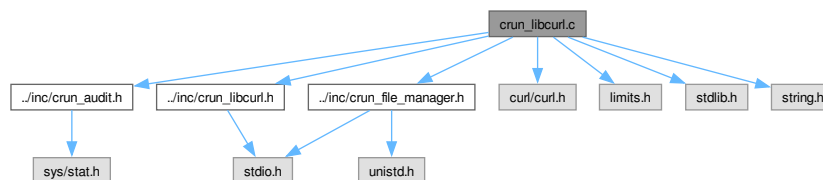
10.23 crun_libcurl.c File Reference

crun_libcurl source file

```

#include "../inc/crun_libcurl.h"
#include "../inc/crun_audit.h"
#include "../inc/crun_file_manager.h"
#include <curl/curl.h>
#include <limits.h>
#include <stdlib.h>
#include <string.h>
  
```

Include dependency graph for crun_libcurl.c:



Classes

- struct [DownloadProgress](#)

Functions

- size_t [write_data](#) (void *ptr, size_t size, size_t nmemb, void *stream)
libcurl write callback used when storing response data to a file.

- int [download_file](#) (const char *url, const char *out)
Download a file from URL to a destination path.

10.23.1 Detailed Description

crun_libcurl source file

Author

ZouariOmar zouariomar20@gmail.com

Version

0.1

Date

2025-12-26

Copyright

Copyright (c) 2025

[crun_libcurl.c](#)

10.23.2 Function Documentation

10.23.2.1 [download_file](#)()

```
int download_file (  
    const char * url,  
    const char * out)
```

Download a file from URL to a destination path.

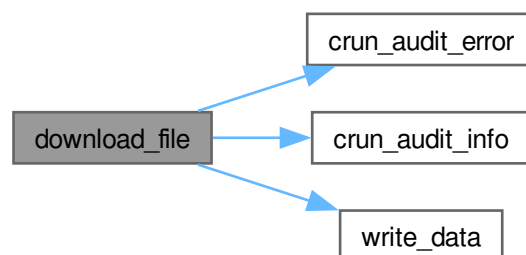
Parameters

url	Remote URL.
out	Destination file path.

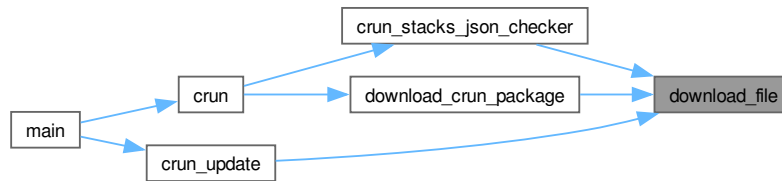
Returns

int EXIT_SUCCESS on success, EXIT_FAILURE otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



10.23.2.2 write_data()

```

size_t write_data (
    void * ptr,
    size_t size,
    size_t nmemb,
    void * stream)

```

libcurl write callback used when storing response data to a file.

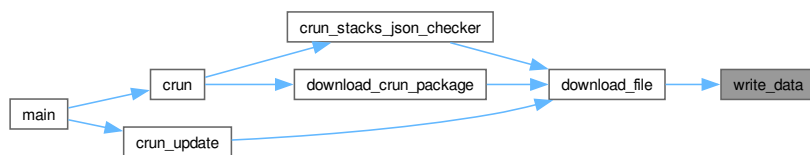
Parameters

ptr	Pointer to downloaded bytes.
size	Size of each element.
nmemb	Number of elements.
stream	Destination stream.

Returns

size_t Number of bytes written.

Here is the caller graph for this function:



10.24 crun_zip_manager.c File Reference

crun_zip_manager source file

```

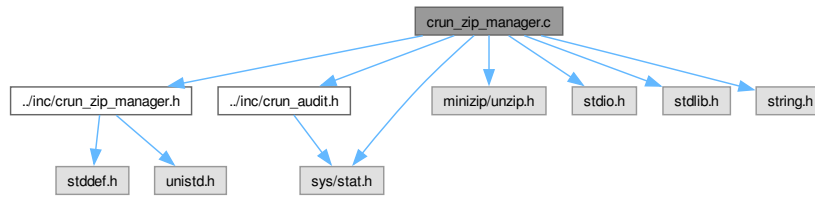
#include "../inc/crun_zip_manager.h"
#include "../inc/crun_audit.h"
#include <minizip/unzip.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```



```
#include <sys/stat.h>
```

Include dependency graph for crun_zip_manager.c:



Functions

- `int extract_zip (const char *zip_path, const char *out_dir, char *init_script_path, size_t init_script_path_size)`
Extract ZIP content safely into output directory.

10.24.1 Detailed Description

crun_zip_manager source file

Author

ZouariOmar zouariomar20@gmail.com

Version

0.1

Date

2026-01-01

Copyright

Copyright (c) 2025

[crun_zip_manager.c README.md](#)

10.24.2 Function Documentation

10.24.2.1 extract_zip()

```
int extract_zip (
    const char * zip_path,
    const char * out_dir,
    char * init_script_path,
    size_t init_script_path_size)
```

Extract ZIP content safely into output directory.

Detects init script path (___init___.sh or ___init___.bat) while extracting.

Parameters

zip_path	Source zip file path.
out_dir	Destination extraction directory.

<code>init_script_path</code>	Optional output buffer to receive detected init script path.
<code>init_script_path_size</code>	Size of <code>init_script_path</code> buffer.

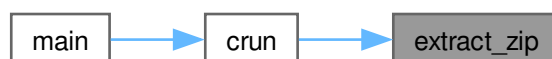
Returns

`int` `EXIT_SUCCESS` on success, `EXIT_FAILURE` otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



10.25 main.c File Reference

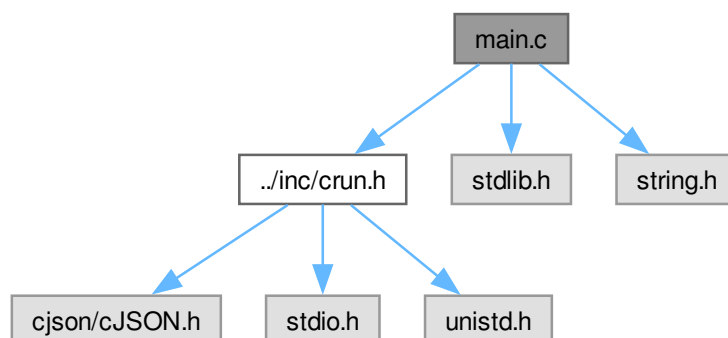
Main source file.

```
#include "../inc/crun.h"
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

Include dependency graph for `main.c`:



Functions

- `int main (int argc, const char **argv)`
CLI entry point with flag dispatch.

10.25.1 Detailed Description

Main source file.

Author

ZouariOmar (zouariomar20@gmail.com)

Version

0.2

Date

2025-11-26

Copyright

Copyright (c) 2025

[main.c](#)

10.25.2 Function Documentation

10.25.2.1 `main()`

```
int main (  
    int argc,  
    const char ** argv)  
CLI entry point with flag dispatch.
```

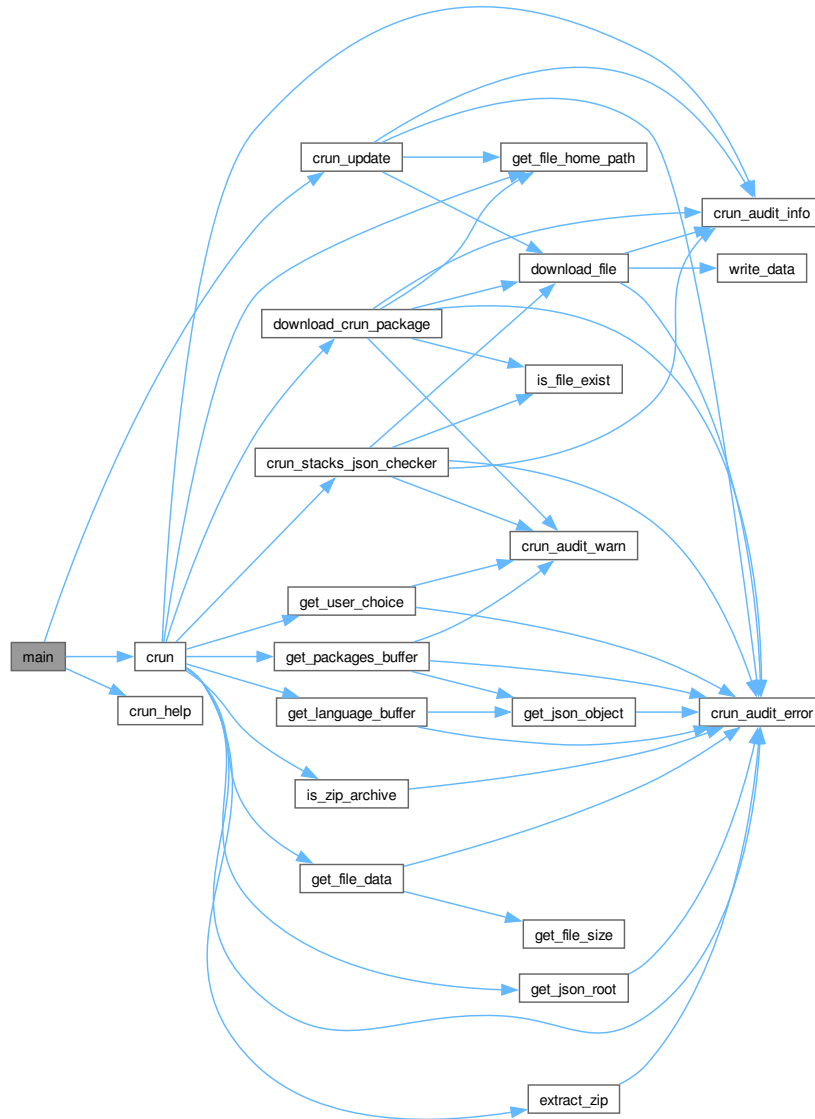
Parameters

<code>argc</code>	Argument count
<code>argv</code>	Argument vector

Returns

int EXIT_SUCCESS/EXIT_FAILURE.

Here is the call graph for this function:



10.26 README.md File Reference

10.27 SECURITY.md File Reference

Index

- [__get_current_dir](#)
 - [crun.h, 34](#)
- [__get_pid](#)
 - [crun.h, 34](#)
- [__mkdir](#)
 - [crun_audit.h, 43](#)
- [bgBlue](#)
 - [color.h, 28](#)
- [bgCyan](#)
 - [color.h, 28](#)
- [bgGreen](#)
 - [color.h, 28](#)
- [bgMagenta](#)
 - [color.h, 28](#)
- [bgRed](#)
 - [color.h, 28](#)
- [bgWhite](#)
 - [color.h, 28](#)
- [bgYellow](#)
 - [color.h, 28](#)
- [black](#)
 - [color.h, 28](#)
- [bleu](#)
 - [color.h, 28](#)
- [CODE_OF_CONDUCT.md, 27](#)
- [color.h, 27, 29](#)
 - [bgBlue, 28](#)
 - [bgCyan, 28](#)
 - [bgGreen, 28](#)
 - [bgMagenta, 28](#)
 - [bgRed, 28](#)
 - [bgWhite, 28](#)
 - [bgYellow, 28](#)
 - [black, 28](#)
 - [bleu, 28](#)
 - [cyan, 29](#)
 - [def, 29](#)
 - [gray, 29](#)
 - [green, 29](#)
 - [magenta, 29](#)
 - [red, 29](#)
 - [white, 29](#)
 - [yellow, 29](#)
- [common Directory Reference, 19](#)
- [Contributing to crun-cli, 7](#)
- [CONTRIBUTING.md, 27](#)
- [Contributor Code of Conduct, 5](#)
- [copy_file_to_path](#)
 - [crun_file_manager.c, 72](#)
 - [crun_file_manager.h, 47](#)
- [crun](#)
 - [crun.c, 62](#)
 - [crun.h, 34](#)
- [crun-cli, 1](#)
- [crun.c, 60](#)
 - [crun, 62](#)
 - [crun_help, 63](#)
 - [crun_stacks_json_checker, 63](#)
 - [crun_update, 64](#)
 - [download_crun_package, 64](#)
 - [free_all, 65](#)
 - [get_language_buffer, 65](#)
 - [get_packages_buffer, 66](#)
 - [get_user_choice, 67](#)
 - [languages_map, 68](#)
 - [languages_map_length, 68](#)
 - [packages_map, 68](#)
 - [packages_map_length, 68](#)
- [crun.h, 32, 41](#)
 - [__get_current_dir, 34](#)
 - [__get_pid, 34](#)
 - [crun, 34](#)
 - [crun_help, 35](#)
 - [crun_stacks_json_checker, 36](#)
 - [crun_update, 36](#)
 - [download_crun_package, 37](#)
 - [free_all, 38](#)
 - [get_language_buffer, 38](#)
 - [get_packages_buffer, 39](#)
 - [get_user_choice, 40](#)
- [crun_audit.c, 68](#)
 - [crun_audit_error, 69](#)
 - [crun_audit_info, 70](#)
 - [crun_audit_warn, 71](#)
- [crun_audit.h, 42, 45](#)
 - [__mkdir, 43](#)
 - [crun_audit_error, 43](#)
 - [crun_audit_info, 44](#)
 - [crun_audit_warn, 45](#)
- [crun_audit_error](#)
 - [crun_audit.c, 69](#)
 - [crun_audit.h, 43](#)
- [crun_audit_info](#)
 - [crun_audit.c, 70](#)
 - [crun_audit.h, 44](#)
- [crun_audit_warn](#)
 - [crun_audit.c, 71](#)

- crun_audit.h, 45
- CRUN_BANNER
 - patterns.h, 31
- CRUN_DEFAULT_SUFFIX_DIRECTORY
 - crun_file_manager.h, 47
- crun_file_manager.c, 71
 - copy_file_to_path, 72
 - CRUN_IO_BUFFER_SIZE, 72
 - get_file_data, 73
 - get_file_home_path, 73
 - get_file_size, 74
 - is_file_exist, 74
 - is_zip_archive, 75
- crun_file_manager.h, 45, 51
 - copy_file_to_path, 47
 - CRUN_DEFAULT_SUFFIX_DIRECTORY, 47
 - CRUN_STACKS_JSON_FILE_DEFAULT_SUFFIX, 47
 - CRUN_STACKS_JSON_FILE_URL, 47
 - CRUN_TMP_DIRECTORY_SUFFIX, 47
 - get_file_data, 48
 - get_file_home_path, 49
 - get_file_size, 49
 - is_file_exist, 49
 - is_zip_archive, 50
 - OS_HOME, 47
- CRUN_HEADER
 - patterns.h, 31
- crun_help
 - crun.c, 63
 - crun.h, 35
- CRUN_IO_BUFFER_SIZE
 - crun_file_manager.c, 72
- crun_json_manager.c, 76
 - get_json_object, 76
 - get_json_root, 77
- crun_json_manager.h, 51, 54
 - get_json_object, 52
 - get_json_root, 53
- crun_libcurl.c, 78
 - download_file, 79
 - write_data, 80
- crun_libcurl.h, 54, 57
 - CURL_CA_CERT_NAME, 55
 - CURL_CA_CERT_REMOTE_PATH, 55
 - download_file, 56
 - write_data, 56
- crun_stacks_json_checker
 - crun.c, 63
 - crun.h, 36
- CRUN_STACKS_JSON_FILE_DEFAULT_SUFFIX
 - crun_file_manager.h, 47
- CRUN_STACKS_JSON_FILE_URL
 - crun_file_manager.h, 47
- CRUN_TMP_DIRECTORY_SUFFIX
 - crun_file_manager.h, 47
- crun_update
 - crun.c, 64
 - crun.h, 36
- crun_zip_manager.c, 80
 - extract_zip, 81
- crun_zip_manager.h, 57, 60
 - extract_zip, 59
 - MKDIR, 59
 - PATH_SEP, 59
- CrunLanguage, 23
 - name, 23
- CrunPackage, 23
 - description, 24
 - name, 24
 - url, 24
- CrunRuntimeContext, 24
 - json_root, 25
 - json_root_buffer, 25
 - languages_buffer, 25
 - packages_buffer, 25
 - stacks_path, 25
 - workspace_path, 25
- CURL_CA_CERT_NAME
 - crun_libcurl.h, 55
- CURL_CA_CERT_REMOTE_PATH
 - crun_libcurl.h, 55
- cyan
 - color.h, 29
- def
 - color.h, 29
- description
 - CrunPackage, 24
- download_crun_package
 - crun.c, 64
 - crun.h, 37
- download_file
 - crun_libcurl.c, 79
 - crun_libcurl.h, 56
- DownloadProgress, 26
 - last_percent, 26
 - started, 26
- extract_zip
 - crun_zip_manager.c, 81
 - crun_zip_manager.h, 59
- free_all
 - crun.c, 65
 - crun.h, 38
- get_file_data
 - crun_file_manager.c, 73
 - crun_file_manager.h, 48
- get_file_home_path
 - crun_file_manager.c, 73
 - crun_file_manager.h, 49
- get_file_size
 - crun_file_manager.c, 74
 - crun_file_manager.h, 49

- get_json_object
 - crun_json_manager.c, [76](#)
 - crun_json_manager.h, [52](#)
- get_json_root
 - crun_json_manager.c, [77](#)
 - crun_json_manager.h, [53](#)
- get_language_buffer
 - crun.c, [65](#)
 - crun.h, [38](#)
- get_packages_buffer
 - crun.c, [66](#)
 - crun.h, [39](#)
- get_user_choice
 - crun.c, [67](#)
 - crun.h, [40](#)
- gray
 - color.h, [29](#)
- green
 - color.h, [29](#)
- HELP_MSG
 - patterns.h, [31](#)
- inc Directory Reference, [19](#)
- INSTALLATION_COMPLETE_MSG
 - patterns.h, [31](#)
- is_file_exist
 - crun_file_manager.c, [74](#)
 - crun_file_manager.h, [49](#)
- is_zip_archive
 - crun_file_manager.c, [75](#)
 - crun_file_manager.h, [50](#)
- json_root
 - CrunRuntimeContext, [25](#)
- json_root_buffer
 - CrunRuntimeContext, [25](#)
- languages_buffer
 - CrunRuntimeContext, [25](#)
- languages_map
 - crun.c, [68](#)
- languages_map_length
 - crun.c, [68](#)
- last_percent
 - DownloadProgress, [26](#)
- magenta
 - color.h, [29](#)
- main
 - main.c, [83](#)
- main.c, [82](#)
 - main, [83](#)
- MKDIR
 - crun_zip_manager.h, [59](#)
- name
 - CrunLanguage, [23](#)
 - CrunPackage, [24](#)
- OS_HOME
 - crun_file_manager.h, [47](#)
- packages_buffer
 - CrunRuntimeContext, [25](#)
- packages_map
 - crun.c, [68](#)
- packages_map_length
 - crun.c, [68](#)
- PATH_SEP
 - crun_zip_manager.h, [59](#)
- patterns.h, [30](#), [32](#)
 - CRUN_BANNER, [31](#)
 - CRUN_HEADER, [31](#)
 - HELP_MSG, [31](#)
 - INSTALLATION_COMPLETE_MSG, [31](#)
 - PROJECT_MENU, [31](#)
 - PROJECT_MENU_FIELD, [31](#)
 - PROJECT_MENU_FIELD_LENGTH, [32](#)
- project Directory Reference, [20](#)
- PROJECT_MENU
 - patterns.h, [31](#)
- PROJECT_MENU_FIELD
 - patterns.h, [31](#)
- PROJECT_MENU_FIELD_LENGTH
 - patterns.h, [32](#)
- README.md, [84](#)
- red
 - color.h, [29](#)
- Security Policy for crun-cli, [11](#)
- SECURITY.md, [84](#)
- src Directory Reference, [21](#)
- stacks_path
 - CrunRuntimeContext, [25](#)
- started
 - DownloadProgress, [26](#)
- url
 - CrunPackage, [24](#)
- white
 - color.h, [29](#)
- workspace_path
 - CrunRuntimeContext, [25](#)
- write_data
 - crun_libcurl.c, [80](#)
 - crun_libcurl.h, [56](#)
- yellow
 - color.h, [29](#)